

Heaven's Light Is Our Guide

Rajshahi University of Engineering & Technology
Department of Computer Science & Engineering



A report on “Traffic-Sentinel”

Course Code: CSE 3200

Course Title: Software Development Project II

Authors

Arittro Sarkar

2103003

Section: A

Supervisor

Dr. Md. Shahid Uz Zaman

Professor

Dept. of CSE, RUET

Date: 07/01/2026

Rajshahi University of Engineering & Technology
Department of Computer Science & Engineering



Certification

This is to certify that the project titled: “**Traffic-Sentinel is a rule-based traffic detection and automated enforcement system designed to detect web attacks, abuse, and automated threats in real time, assign penalties and apply firewall-based bans**” submitted by *Arittro Sarkar*, Roll Number(s) *2103003*, for the fulfillment of the requirements of the course CSE 3200 (Software Development Project II), has been examined and approved by the undersigned.

The project has been carried out under my supervision, and it is recommended for submission and evaluation.

Supervisor:

Name: Dr. Md. Shahid Uz Zaman

Designation: Professor

Department: CSE

Institution: Rajshahi University of Engineering & Technology

Date: _____

Signature: _____

Contents

	Page No.
Abstract	05
Chapters	
1. Introduction	08-10
1.1 Problem Statement	
1.2 Motivation	
1.3 Objectives	
1.4 Contribution Summary	
1.5 Report Structure	
2. Background & Literature Review	11-16
2.1 Theoretical Foundation	
2.2 Related Work Review	
2.3 Comparative Analysis	
2.4 Gap Analysis	
3. Requirements & Team Workflow	17-19
3.1 Functional Requirements	
3.2 Non-functional Requirements	
3.3 System Constraints	
3.4 Technology Stack Justification	
3.5 Team Structure & Individual Responsibilities	
3.6 Collaboration Workflow	
4. Project Management & Finance	20-25
4.1 Work Breakdown Structure (WBS)	
4.2 Project Schedule — Gantt Chart	
4.3 Budget & Financial Cost Analysis	
4.4 Risk Analysis & Mitigation Plan	
4.5 Summary of Management Decisions	
5. System Design & Architecture	26-28
5.1 Methodological Structure	
5.2 High-Level Architecture Diagram	
5.3 Module Interaction Diagram	
5.4 Database Schema / ER Diagram	
5.5 Interface Specifications / API Contracts	
5.6 Security & Performance Design	
6. Implementation	29-40

6.1	Development Environment Setup	
6.2	Coding Standards & Version Control Strategy	
6.3	Module-Wise Implementation	
6.3.1	Module Ownership Block	
6.4	System Integration Workflow	
6.5	Performance, Optimization, and Security Techniques	
7.	Testing, Results & Evaluation	41-46
7.1	Testing Strategy	
7.2	Test Case Tables	
7.3	Performance Evaluation	
7.4	Comparison with Baseline or Existing Systems	
7.5	Screenshots & Execution Results	
8.	Discussion & Analysis	47-49
8.1	Interpretation of Findings	
8.2	Strengths of the Project	
8.3	Limitations	
8.4	Recommendations for Future Development	
8.5	Reflection on Design & Implementation Decisions	
9.	Life-Long Learning impact	50-52
9.1	Technical & Professional Skills Acquired	
9.2	Learning New Technologies & Tools	
9.3	Future Growth & Directions	
10.	Conclusion	53-54
10.1	Summary of Work	
10.2	Achievement of Objectives	
10.3	Final Remarks	
	REFERENCES	55
	APPENDICES	56-58

Abstract

Traffic-Sentinel is a lightweight, log-based web traffic detection and automated enforcement system designed to identify malicious HTTP and HTTPS activity such as scanning, exploit probing, brute-force attacks, and abusive automated requests in real time. Modern web infrastructures often rely on heavyweight Web Application Firewalls (WAFs) or opaque machine-learning models, which can be resource-intensive, difficult to audit, and challenging to integrate with existing security pipelines. Traffic-Sentinel addresses these limitations by adopting a deterministic, rule-based approach that emphasizes transparency, explain ability and low operational overhead.

The system operates primarily in log ingestion mode, continuously analyzing structured JSONL access logs generated by web servers and reverse proxies such as Nginx or Apache. Since HTTPS traffic is terminated at the web server or proxy layer, Traffic-Sentinel retains full application-layer visibility without requiring packet-level inspection. Incoming requests are normalized and evaluated against curated, high-signal detection rules based on regular expressions and contextual analysis. Each suspicious request contributes to a per-IP reputation score, allowing the system to distinguish isolated noise from persistent or high-risk attackers. Additional logic is implemented for detecting authentication brute-force attempts by correlating request frequency and HTTP response codes over time. Detected events are written to structured, append-only logs and can optionally trigger real-time alerts via Telegram or be consumed by external security tools such as Fail2Ban, SIEM platforms, or custom automation scripts.

Traffic-Sentinel is designed to be safe and predictable by default, performing no inline request blocking and no mandatory firewall enforcement, while still providing all necessary data for downstream mitigation. Its modular architecture, Docker-friendly deployment, and minimal CPU and memory footprint make it suitable for single-host and small-scale production environments. The primary contribution of this project lies in demonstrating an efficient, explainable, and integration-friendly alternative to traditional web security solutions, offering clear visibility into application-layer attacks without sacrificing performance or control.

List of Figures

	Page No.
Gantt Chart Illustrating the Development Schedule of Traffic-Sentinel	22
Methodological Pipeline of Traffic-Sentinel	26
High-Level System Architecture of Traffic-Sentinel	26
Module Interaction Sequence in Traffic-Sentinel	27
Conceptual ER / Data Schema of Traffic-Sentinel	27
Interface Specifications and Integration Contracts of Traffic-Sentinel	28
Security and Performance Design Overview of Traffic-Sentinel	28
Git commit history from the master branch showing incremental development of the Traffic Sentinel project	39
System Integration Workflow of Traffic-Sentinel (module-to-module data flow from ingestion to enforcement and alerting)	40
Detection of Cross-Site Scripting (XSS) Attack	44
Detection of SQL Injection Attack	45
Real-Time Telegram Alert Triggered by Repeated Malicious Activity	46

List of Tables

Page No.

Core Objectives of the Traffic-Sentinel System	9
Estimated Budget for Traffic-Sentinel (all major items)	23
Development Environment Configuration for Traffic-Sentinel	29
Security Test Cases Showing Expected and Actual Detection Outputs	42
Performance Evaluation Metrics of Traffic-Sentinel	43

Chapter 1

Introduction

1.1 Problem Statement

Modern web applications are continuously exposed to malicious traffic such as automated scanning, exploit probing, brute-force login attempts, and abusive scripted requests. These threats often originate from distributed sources and may individually appear benign, making detection difficult when relying solely on traditional security mechanisms. Conventional solutions such as Web Application Firewalls (WAFs) are frequently heavyweight, opaque in decision-making, resource-intensive, and tightly coupled with inline traffic blocking, which can lead to false positives and service disruption. Additionally, machine-learning-based systems often lack explainability and require large datasets, making them unsuitable for small-scale or resource-constrained deployments.

There exists a need for a lightweight, explainable, and log-driven detection system that can analyze real web traffic both HTTP and HTTPS without interfering directly with request handling, while still providing reliable detection, scoring, and actionable outputs for downstream enforcement. The absence of such a transparent and integration-friendly solution motivates the development of Traffic-Sentinel.

1.2 Motivation

The increasing adoption of HTTPS has rendered packet-level inspection ineffective without complex decryption setups, shifting security visibility toward the application and log layer. Web servers such as Nginx already generate rich access logs containing decrypted request data, yet this information is often underutilized for real-time security analysis.

The motivation behind Traffic-Sentinel is to leverage these structured logs to build a deterministic and auditable security monitoring system that operates independently of traffic flow. By focusing on rule-based detection rather than probabilistic models, the system ensures predictable behavior and easy interpretability. Furthermore, many administrators require security tools that integrate cleanly with existing ecosystems such as Fail2Ban, SIEM platforms, or custom scripts, without enforcing a rigid security model. Traffic-Sentinel is designed to fill this gap by prioritizing visibility, modularity, and operational simplicity.

1.3 Objectives

Table 1.3: Core Objectives of the Traffic-Sentinel System

Objective Name	Description
Malicious Request Detection	To detect malicious HTTP and HTTPS requests using structured web server logs.
Explainable Rule-Based Analysis	To implement a deterministic, rule-based detection engine with clearly interpretable outcomes.
IP Reputation and Scoring	To assign per-IP reputation scores based on cumulative malicious behavior over time.
Bruteforce Attack Identification	To detect authentication brute-force attempts through temporal analysis of access logs.
Structured Security Event Generation	To generate structured, append-only security event logs for auditing and analysis.
Real-Time Alerting Support	To support optional real-time alerting through Telegram notifications.
Resource Efficient Operation	To maintain low CPU and memory usage suitable for single-host deployments.
External Tool Integration	To enable seamless integration with external enforcement and monitoring tools such as Fail2Ban and SIEM systems.

1.4 Contribution Summary

This project makes the following key contributions:

1. Log-Centric Security Model:

A security detection pipeline that operates entirely on application-layer logs, ensuring full compatibility with HTTPS traffic.

2. Explainable Rule-Based Detection Engine:

Curated regex-based rules grouped by attack category, ensuring transparency and preventing score inflation through controlled matching.

3. Per-IP Reputation and Scoring Mechanism:

A cumulative scoring model that distinguishes persistent attackers from isolated anomalies.

4. Bruteforce Detection via Log Correlation:

Detection of authentication abuse based on request rate, endpoint targeting, and HTTP response codes.

5. Modular and Integration-Friendly Architecture:

Structured outputs suitable for Fail2Ban, SIEM ingestion, alerting systems, or custom automation.

6. Safe-by-Design Enforcement Strategy:

No mandatory inline blocking, reducing operational risk while preserving enforcement flexibility.

1.5 Report Structure

This report is organized into ten chapters, each addressing a specific aspect of the Traffic-Sentinel project in a systematic manner.

Chapter 1: introduces the project by defining the problem addressed, outlining the motivation, stating the objectives, summarizing the contributions, and explaining the overall structure of the report.

Chapter 2: presents the background and literature review, including the theoretical foundations of web traffic security, a review of related work, a comparative analysis of existing solutions, and an identification of research and implementation gaps addressed by this project.

Chapter 3: describes the project requirements and team workflow. It details the functional and non-functional requirements, system constraints, justification of the selected technology stack, team roles and responsibilities, and the collaboration workflow followed during development.

Chapter 4: focuses on project management and financial planning. It includes the work breakdown structure, project scheduling through a Gantt chart, budget and cost analysis, risk assessment with mitigation strategies, and a summary of key management decisions.

Chapter 5: explains the system design and architecture of Traffic-Sentinel. This chapter covers the methodological structure, high-level architecture, module interaction diagrams, data structures or schema design, interface specifications, and security and performance considerations.

Chapter 6: details the implementation phase of the project. It discusses the development environment setup, coding standards and version control strategy, module-wise implementation with ownership mapping, system integration workflow, and applied performance optimization and security techniques.

Chapter 7: presents testing, results, and evaluation. It outlines the testing strategy, test cases, performance evaluation results, comparison with baseline or existing systems, and relevant execution outputs.

Chapter 8: provides a discussion and analysis of the project outcomes, including interpretation of results, strengths and limitations of the system, recommendations for future development, and reflections on design and implementation decisions.

Chapter 9: highlights the life-long learning impact of the project, focusing on the technical and professional skills acquired, new technologies and tools learned, and directions for future personal and professional growth.

Chapter 10: concludes the report by summarizing the overall work, evaluating the achievement of project objectives, and presenting final remarks.

Chapter 2

Background & Literature Review

2.1 Theoretical Foundation

HTTP & HTTPS

HTTP (HyperText Transfer Protocol) is a stateless application-layer protocol used for client–server communication on the web. HTTPS is HTTP secured using TLS/SSL, ensuring confidentiality, integrity, and authentication.

TLS / SSL

TLS (Transport Layer Security) encrypts data exchanged between clients and servers, preventing eavesdropping and tampering. In practical deployments, TLS often terminates at a reverse proxy (e.g., Nginx).

Web Server / Reverse Proxy

A web server serves client requests, while a reverse proxy sits in front of backend services to handle routing, TLS termination, and logging.

Nginx

Nginx is a high-performance web server and reverse proxy widely used for logging HTTP traffic in structured formats such as JSON.

JSONL (JSON Lines)

JSONL stores one JSON object per line, making it ideal for real-time log streaming and parsing.

Example:

```
{"ts":"2025-12-27T16:58:41Z","uri":"/wp-login.php","status":404}
```

Log Tailing / Polling

Log tailing continuously monitors logs as they grow, while polling checks files at fixed intervals.

Traffic-Sentinel uses log tailing to enable near real-time detection.

Log Rotation (logrotate)

Log rotation prevents log files from growing indefinitely. Traffic-Sentinel is designed to handle rotated logs without losing detection continuity.

HTTP Headers

Headers carry metadata about the request and client. Key headers analyzed:

User-Agent: Identifies the client software. Unusual or empty user-agents often indicate automated attack tools.

Referrer: Indicates the source of a request. Missing or suspicious referrers can help identify bot traffic.

Content-Type: Defines the format of request data. Example: application/x-www-form-urlencoded

X-Forwarded-For: Used to preserve the real client IP when traffic passes through proxies or CDNs (e.g., Cloudflare). Traffic-Sentinel relies on XFF to accurately identify attackers.

IPv4 & IPv6

Internet addressing schemes used to identify clients and servers. Traffic-Sentinel supports detection across both IP versions.

CIDR

Classless Inter-Domain Routing defines IP ranges. Example: 192.168.1.0/24 allows grouping and banning attacker subnets.

Firewall

A firewall controls incoming and outgoing network traffic based on predefined rules.

Iptables

iptables is a Linux firewall utility used to block malicious IPs detected by Traffic-Sentinel.

PCAP (Packet Capture)

PCAP files store raw network packets for offline or real-time analysis.

tcpdump & tshark

Command-line tools used for capturing and analyzing packets.

BPF Filter

Berkeley Packet Filter enables efficient packet filtering at capture time. Example: tcp port 80

SYN / TCP Flags

TCP flags control connection states.

SYN Detection & SYN Flood

Repeated SYN packets without completed handshakes indicate SYN flood attacks. Traffic-Sentinel detects these using metadata without full packet payload inspection.

Docker

Docker provides isolated, reproducible environments for deployment. Consistent execution, Easy deployment, Service isolation.

XSS (Cross-Site Scripting)

Cross-Site Scripting occurs when an attacker injects malicious JavaScript into a web page, which is then executed in the victim's browser.

Example: GET /search?q=<script>alert(1)</script>

SQL Injection (SQLi)

SQL Injection manipulates backend database queries by injecting malicious SQL code through unsanitized user input.

Example:

POST /login

username=admin' OR '1'='1

LFI (Local File Inclusion)

LFI allows attackers to read local server files by manipulating file inclusion parameters.

Example:

GET /index.php?page=../../../../etc/passwd

Path Traversal

Path traversal exploits directory navigation to access restricted files.

Example:

GET /download?file=../config/settings.conf

Null Byte Injection

Uses a null byte (%00) to bypass file extension validation mechanisms.

Example:

GET /load?file=shell.php%00.jpg

RCE (Remote Code Execution)

Remote Code Execution allows attackers to run arbitrary code on the server.

Example:

GET /vuln.php?cmd=id

Command Injection & Subshell Execution

Attackers inject operating system commands into application input fields.

Example:

GET /ping?host=127.0.0.1; whoami

SSRF (Server-Side Request Forgery)

SSRF forces the server to make requests to internal or restricted resources.

Example:

GET /fetch?url=http://127.0.0.1/admin

SSTI & Jinja Template Injection

Server-Side Template Injection occurs when user input is evaluated inside a template engine.

Example (Jinja):

```
{{7*7}}
```

XXE (XML External Entity)

XXE exploits insecure XML parsers to read internal files or initiate network requests.

Example:

```
<!DOCTYPE data [  
  <!ENTITY xxe SYSTEM "file:///etc/passwd">  

```

PHP Attack Patterns & Stream Wrappers

PHP stream wrappers allow access to special input/output streams that can be abused.

Examples:

```
php://filter/convert.base64-encode/resource=index.php
```

```
php://input
```

Webshell Probe

Automated attempts to locate uploaded malicious shell scripts.

Example:

```
GET /uploads/shell.php
```

WordPress Scanning

Automated probing of known WordPress paths to find vulnerabilities.

Example:

```
GET /wp-login.php
```

```
GET /wp-admin/
```

Dotfile Scanning

Attempts to access hidden configuration files.

Example:

```
GET /.env
```

```
GET /.git/config
```

Bruteforce Detection

Bruteforce attacks involve repeated authentication attempts against login endpoints using different username–password combinations. The objective is to guess valid credentials by exploiting weak or reused passwords.

2.2 Related Work Review

Web traffic monitoring and intrusion detection have been extensively studied, with existing solutions broadly categorized into network-based intrusion detection systems (NIDS), web application firewalls (WAFs), and log-based security monitoring tools. Traditional NIDS such as Snort and Suricata analyze packet-level traffic using signature-based and rule-based approaches, providing deep inspection but often requiring high computational resources and complex configurations. Similarly, Web Application Firewalls like ModSecurity operate at the application layer, intercepting HTTP requests in real time; however, they can introduce latency and are prone to false positives when deployed with generic rule sets.

Recent research has explored machine learning–based intrusion detection systems that model normal traffic behavior and detect anomalies. While these approaches improve adaptability, they often require large labeled datasets, frequent retraining, and lack transparency in decision-making. In contrast, log-based detection systems leverage structured web server logs to identify malicious patterns without direct traffic interception. Tools such as Fail2Ban demonstrate the effectiveness of rule-based log analysis for brute-force mitigation but are limited in attack coverage and extensibility.

Traffic-Sentinel builds upon log-based detection principles while extending beyond single-attack focus systems. By combining deterministic rule evaluation, request pattern analysis, and subnet-level aggregation, it provides a lightweight yet expressive detection framework suitable for modern web infrastructures.

2.3 Comparative Analysis

Existing web attack detection approaches differ significantly in terms of deployment complexity, detection depth, and operational overhead. Packet-based intrusion detection systems such as Snort and Suricata provide deep traffic inspection and high detection accuracy through signature matching. However, these systems require access to raw network traffic, specialized configurations, and substantial computational resources, making them difficult to deploy in resource-constrained or containerized environments.

Web Application Firewalls operate at the application layer and offer real-time request blocking. While effective against known attack signatures, WAFs often suffer from high false-positive rates and may disrupt legitimate user traffic. Additionally, tuning WAF rules for specific applications requires continuous manual effort and domain expertise.

Machine learning–based detection systems aim to identify unknown attack patterns through behavioral modeling. Although promising, these systems rely heavily on quality training data, are computationally expensive, and often lack explainability, which limits their adoption in security-critical environments. In contrast, log-based security tools such as Fail2Ban provide lightweight detection by analyzing server logs but are typically limited to brute-force attacks and lack extensibility for complex web attack patterns. These limitations highlight the need for a flexible, low-overhead detection approach that balances accuracy, transparency, and deployment simplicity.

2.4 Gap Analysis

Despite extensive research in web intrusion detection, several gaps remain in existing approaches. Packet-based intrusion detection systems provide deep analysis but demand high computational resources and direct access to network traffic, which is often unavailable in cloud-native or reverse-proxy-based deployments. Web Application Firewalls offer real-time protection but require continuous rule tuning and may introduce latency or false positives that affect legitimate users. Machine learning-based systems improve adaptability but depend on large, high-quality datasets and lack transparency in decision-making, making them difficult to audit and trust in security-sensitive environments.

Log-based tools, while lightweight and easy to deploy, are typically limited to detecting a narrow set of attacks such as brute-force attempts and do not provide comprehensive coverage of modern web application threats. Furthermore, existing solutions often fail to correlate multi-stage attacks or aggregate behavior across subnets.

These limitations reveal the need for a deterministic, extensible, and log-driven detection framework that can operate efficiently in modern web infrastructures without deep packet inspection or complex training requirements—an area addressed by Traffic-Sentinel.

Chapter 3

Requirements & Team Workflow

3.1 Functional Requirements

This section describes the core functionalities that Traffic-Sentinel must provide to meet its intended objectives.

The system shall continuously monitor web server access logs (JSONL format) generated by Nginx or similar reverse proxies. It shall parse and normalize incoming log entries in near real time to extract relevant request attributes such as IP address, URI, HTTP method, status code, headers, and timestamps. Traffic-Sentinel shall detect malicious HTTP and HTTPS requests using predefined rule-based detection logic. These rules shall cover common web attack patterns including scanning attempts, SQL injection, cross-site scripting (XSS), file inclusion, command injection, brute-force authentication attempts, and automated bot activity.

The system shall maintain a per-IP reputation score by aggregating detected malicious events over time. Based on configurable thresholds, it shall identify persistent or high-risk attackers.

Traffic-Sentinel shall generate structured, append-only security event logs that record detection results, attack categories, scores, and timestamps. It shall support optional real-time alerting through Telegram notifications.

The system shall provide an administrative command-line interface (CLI) to query IP status, view top offenders, clear penalties, unban IPs, and manage allowlists.

3.2 Non-functional Requirements

Traffic-Sentinel shall operate with low CPU and memory overhead to ensure suitability for single-host and small-scale deployments. The system shall be capable of processing high-frequency log streams without introducing noticeable performance degradation.

The detection process shall be deterministic and explainable. Each alert or score increment must be traceable to a specific rule match, ensuring transparency and auditability.

The system shall be reliable and fault-tolerant, continuing to operate correctly across log rotations and temporary log file interruptions.

Traffic-Sentinel shall be secure by design, performing no inline traffic blocking by default and avoiding any modification of live HTTP request handling. It shall ensure data integrity of generated logs and alerts.

The system shall be portable and deployable in containerized environments using Docker.

3.3 System Constraints

Traffic-Sentinel is fundamentally designed as a log-driven detection system and therefore relies on the availability of structured access logs generated by a web server or reverse proxy such as Nginx or Apache. The accuracy and effectiveness of detection are directly dependent on the completeness, consistency, and correctness of these logs. In environments where logging is disabled, misconfigured, or lacks sufficient request detail, the system's visibility and detection capability may be reduced.

The system does not inspect raw network packet payloads by default. Although optional PCAP-based analysis can be enabled, Traffic-Sentinel primarily operates at the application layer using decrypted request data extracted from server logs. As HTTPS traffic is encrypted during transit, the system requires TLS termination at the web server or proxy layer in order to access request information. Deployments without TLS termination at this level cannot be analyzed effectively.

Traffic-Sentinel is designed for Linux-based operating systems and requires read access to log directories and write access for event output files. Optional automated enforcement depends on external firewall tools such as iptables or Fail2Ban, which must be available and properly configured on the host system.

Finally, as a deterministic rule-based system, Traffic-Sentinel's detection coverage is constrained by the quality, scope, and maintenance of its defined rules. The system does not perform autonomous learning or adaptive model training and therefore cannot automatically detect previously unseen attack patterns without manual rule updates.

3.4 Technology Stack Justification

Python was selected as the primary implementation language due to its extensive ecosystem for log processing, regular expression matching, and rapid development of security-focused applications. Its readability and maintainability also support transparent and auditable detection logic, which is a key design goal of Traffic-Sentinel.

Nginx was chosen as the reference web server and reverse proxy because of its widespread use in modern web infrastructures, high performance under heavy traffic, and strong support for structured JSON logging. This makes it well suited for real-time log ingestion and analysis.

The JSON Lines (JSONL) format was selected for log storage as it enables efficient streaming, incremental parsing, and low-latency processing of individual log entries. This format also integrates easily with external monitoring tools and security pipelines.

Docker was used to containerize the system, ensuring deployment consistency, service isolation, and ease of replication across environments. Containerization also simplifies testing and operational management.

Telegram was selected as the alerting platform due to its lightweight API, reliability, and ease of real-time notification integration.

Linux was chosen as the target operating system because of its native support for server workloads, firewall tooling, log management utilities, and production-grade stability.

3.5 Team Structure & Individual Responsibilities

This project was developed by a single member. **`Arittro Sarkar (Roll: 2103003)`**

1. Designed the overall system architecture
2. Implemented log ingestion, parsing, and normalization modules
3. Developed rule-based detection logic for multiple web attack categories
4. Implemented IP reputation scoring and brute-force detection mechanisms
5. Designed structured event logging and alerting system
6. Developed administrative CLI tools
7. Conducted testing, performance evaluation, and documentation

All phases of analysis, design, implementation, testing, and reporting were carried out by the same developer.

3.6 Collaboration Workflow

Since Traffic-Sentinel was developed as a single-developer project, collaboration was managed through a structured and disciplined individual workflow rather than multi-member coordination. Development activities were planned and executed using clearly defined milestones corresponding to major system components.

The project was divided into incremental phases including log ingestion and parsing, rule-based detection engine development, IP reputation scoring, brute-force detection, alerting integration, and deployment configuration. Each phase was implemented sequentially and validated through manual testing using real web server access logs.

Version control was managed using Git, with frequent and descriptive commits to track development progress, facilitate debugging, and ensure rollback capability in case of errors. Modular coding practices were followed to maintain separation of concerns and improve maintainability.

Docker-based testing was used to verify consistent system behavior across environments and configurations. Documentation and report writing were carried out in parallel with implementation to ensure alignment between system functionality and its formal description.

This structured workflow ensured steady progress, reduced integration issues, and maintained traceability between design decisions and implementation outcomes.

Chapter 4

Project Management & Finance

4.1 Work Breakdown Structure (WBS)

The development of Traffic-Sentinel was organized into a series of sequential phases, each corresponding to a major component or milestone of the project. Below is a hierarchical breakdown of the work:

Phase 1: Planning & Requirements Analysis – In the initial stage, I conducted background research on web attack detection approaches and gathered requirements. This included studying existing WAF solutions and log analysis techniques, defining the problem scope and identifying key objectives for the system (e.g. real-time detection, rule-based logic, low overhead). The outcome of this phase was a clear project plan and list of features to implement (detailed in Chapter 1 and 2 of the report).

Phase 2: System Design – Next, the overall architecture and technology stack were designed. I decided to adopt a log-driven architecture (analyzing web server logs post-HTTPS termination) instead of inline packet inspection, to simplify deployment and avoid dealing with encrypted traffic. High-level system components were identified (log ingestor, parser, rule engine, scoring module, alert dispatcher, etc.), and detection rules categories were defined (scans, exploits, brute-force, etc.). During this phase, the technology choices were made: Python for implementation (due to its rich regex and log-processing libraries), JSONL format for logs, and Docker for deployment, among others. The outcome was an architecture plan (later visualized in the architecture diagram) and configuration of development environment (setting up the Git repository and necessary tools).

Phase 3: Core Implementation – The main development work was broken into sub-tasks corresponding to core modules:

- **Log Ingestion & Parsing:** Implemented the module to tail and read JSONL access logs (from Nginx/Apache) and normalize each HTTP request record into a standard structure for analysis. This involved writing the ingestor component (file tailer and JSON parser) and ensuring it could continuously stream log entries in near-real-time.
- **Rule-Based Detection Engine:** Developed the detection logic using deterministic pattern matching. Regex-based rules were created for categories like scanning, PHP exploits, webshell paths, etc., encapsulated in an analyzer module. The system was designed to evaluate each request against the rule set and identify malicious indicators with an explainable rule match (only the best matching rule per request is reported to avoid score inflation).
- **IP Reputation Scoring:** Built a scoring mechanism to accumulate a “reputation” score per IP address based on detected malicious events. This was implemented in an in-memory state (with

periodic writes to JSON for persistence) that increments scores for repeat offenders. No decay or machine learning was used, so this module simply tallied rule hits for each IP.

- **Brute-Force Attack Detection:** Added a specialized sub-module to detect brute-force login attempts and high-frequency attacks by tracking repeated requests with certain statuses or to certain endpoints. This works alongside the rule engine by looking for patterns like many login failures from the same IP.
- **Event Logging & Allowlisting:** Developed components to record security events and decisions. Every detection triggers an event entry (saved in an append-only log file for audit) and updates the in-memory reputation. An allowlist mechanism was also included (via an allowlist module) to exclude known safe IPs or hosts from scoring, preventing false positives from trusted services.
- **Alerting System (Telegram Integration):** Integrated a notification module to send out alerts for high-severity events. A Telegram bot API was used to push real-time alerts to a configured chat/channel, providing immediate insight when an IP exceeds a threshold or a critical rule is triggered. This feature was optional and configurable, but it added a proactive warning mechanism to the tool.
- **Command-Line Interface (CLI):** Implemented an administrative CLI (cli.py) to control the application (for example, a run command to start monitoring). This made the tool user-friendly to operate and test from the console, encapsulating the main runtime loop and configuration loading in a convenient interface.

Phase 4: Integration & Deployment Configuration – Once core features were implemented, the project focused on making the system easy to deploy in real-world environments. The developer containerized the application using Docker, creating a Dockerfile and a docker-compose.yml for running Traffic-Sentinel along with a sample Nginx setup. This ensures consistency across different machines and simplifies dependencies. Additionally, a systemd service unit (trafficsentinel.service) and log rotation script were provided (in the deploy/ folder) to facilitate deploying the tool on a Linux server as a background service. During this phase, the integration between Traffic-Sentinel and the web server was tested – e.g. configuring Nginx to produce the required JSONL logs (with necessary fields like request URI, user-agent, etc.) as per the project’s needs. By the end of this stage, one could easily install Traffic-Sentinel on a server (or run it in Docker) and have it monitor live traffic logs.

Phase 5: Testing & Refinement – Extensive testing was conducted to validate each module’s effectiveness and the overall system performance. The developer used real or simulated web access logs to exercise the detection rules for various attack scenarios (scans, injection attempts, brute-force, etc.), checking that malicious activities were correctly flagged and scored. During testing, some bugs and logical faults were identified and fixed (e.g., a logical error in the main loop was corrected in an update) – this is evidenced by commit messages like “Update some logical fault in the file” and others addressing main logic and config adjustments. The rule set was also refined: patterns causing false positives were adjusted, and obsolete or overly broad rules (including an attempt to import OWASP CRS rules) were removed to

improve signal-to-noise. Notably, an early PCAP capture feature that was prototyped was disabled/removed at this stage, as it proved outside the project scope and not necessary for the log-based approach. This simplification reduced complexity and potential performance overhead. Overall, this phase focused on ensuring the system runs reliably (continuous log processing without crashes), uses minimal resources, and produces accurate detections.

Phase 6: Documentation & Finalization – In the final phase, all project deliverables were completed. The developer prepared comprehensive documentation, including a detailed README explaining system operation, configuration, and limitations. An architecture diagram was created (as an SVG in the docs) to illustrate how Traffic-Sentinel processes data from log ingestion to alert dispatch. Throughout development, documentation was updated in parallel with coding to reflect the latest design – ensuring consistency between the implementation and written description. Finally, the formal project report (including this Chapter 4) was written, summarizing the work and outcomes. All management decisions, design justifications, and results were recorded, and the project was submitted on schedule.

4.2 Project Schedule — Gantt Chart

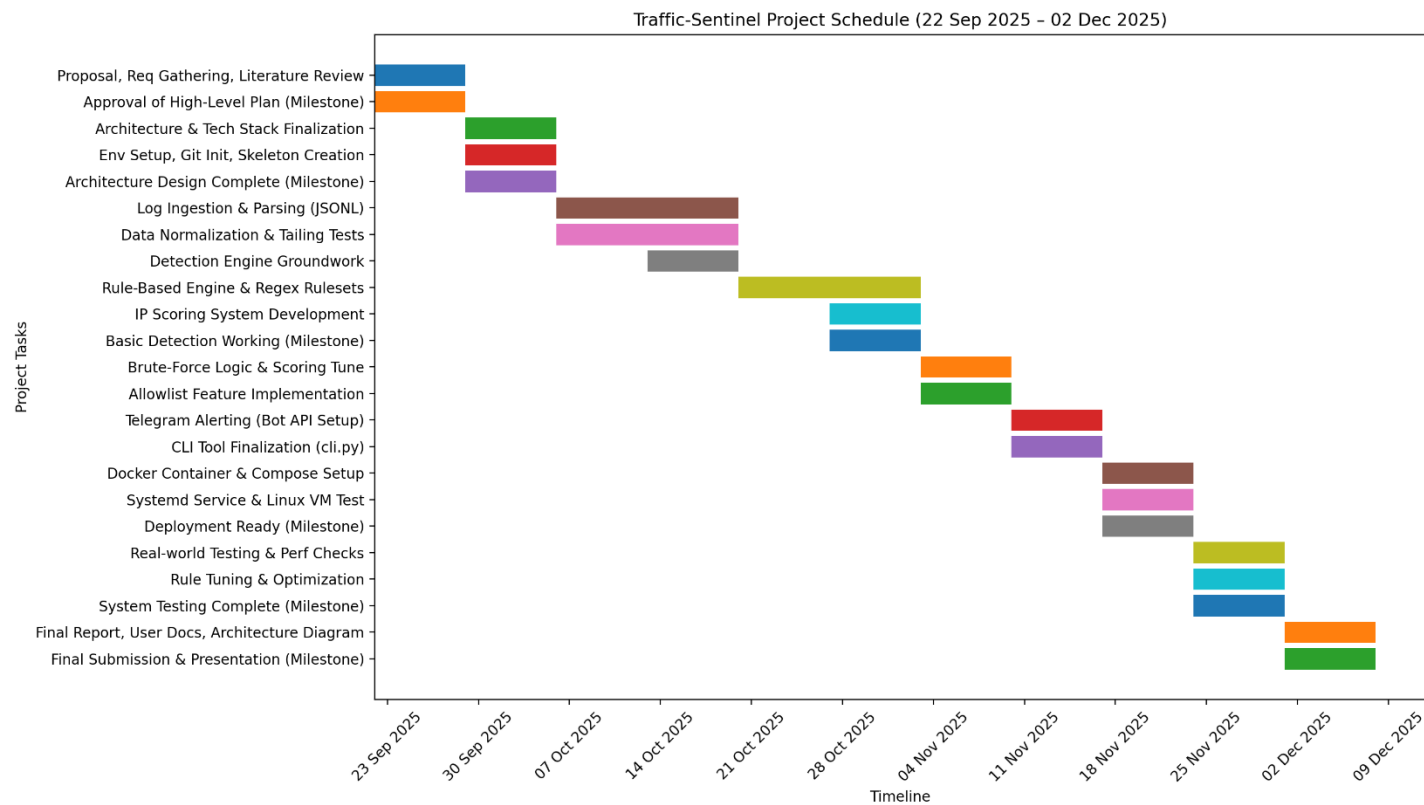


Figure 4.1: Gantt Chart Illustrating the Development Schedule of Traffic-Sentinel

4.3 Budget & Financial Cost Analysis

Traffic-Sentinel was developed as an academic project and thus incurred minimal direct costs. There were no special hardware purchases or paid software licenses required just the server take a budget. This section details the cost considerations:

Item	Cost	Notes
Software Licenses	\$0	All open-source (Python, Nginx, etc.)
Hardware	\$0	Used existing PC or hardware.
Cloud VM (Testing)	\$7.35	Used Server from Digital Ocean.
Alerting Service	\$0	Telegram API used (free)
Miscellaneous	\$0	(No domain, no paid APIs)
Total Cost	\$7.35	

Table 4.1: Estimated Budget for Traffic-Sentinel (all major items)

4.4 Risk Analysis & Mitigation Plan

Developing and deploying a security tool like Traffic-Sentinel involves several risks. Here we identify key project risks and the mitigation strategies employed:

Reliance on Log Availability (Risk): Traffic-Sentinel’s detection capability hinges on the availability and quality of web server logs. If logging is turned off, misconfigured, or missing important details (e.g., failing to record client IPs or request paths), the system cannot detect attacks. This risk is heightened in environments where logs might be rotated too quickly or not retained. Mitigation: The project mandates a structured JSON logging setup on the web server side (detailed Nginx configuration was provided to ensure all relevant fields are logged). The system constraints explicitly note this dependency so that deployers configure logging properly. Additionally, Traffic-Sentinel was tested with the recommended log format to verify it captures necessary data. Monitoring of log integrity (e.g., alerts if log ingestion stops) can be implemented to detect if logging breaks.

Detection Coverage and Evolving Threats (Risk): As a deterministic rule-based system, Traffic-Sentinel can only detect patterns that its rules account for. Novel or obfuscated attacks that do not match any regex rule will go unnoticed. This also means there is no learning of new threats (no AI/ML), and the effectiveness is constrained by the rule set’s completeness. Mitigation: The project addressed this by focusing on well-known, high-impact attack patterns in the initial rule set (ensuring at least common threats are covered). To manage the risk of new threats, the system is designed to be easily updatable – new rules can be added quickly (rules are loaded from configurable files). In an operational setting, the mitigation is to continuously maintain the rules: monitor security bulletins and traffic trends, then update the regex patterns accordingly. The report also recommends considering future enhancements like anomaly detection or periodic rule updates to broaden coverage.

False Positives and Tuning (Risk): Rule-based systems can sometimes flag legitimate user behavior as malicious (false positives) if regex patterns are too broad or aggressive. For example, an odd but harmless URL might match a regex for a webshell path. False positives can erode trust in the system and cause

unnecessary administrative work. Mitigation: During development, careful attention was paid to crafting high-signal rules – patterns chosen were specific to known malicious behavior to minimize accidental matches. Testing with real traffic helped adjust thresholds and patterns. The inclusion of an allowlist feature is a direct mitigation: safe IPs or known friendly services can be exempted if they trigger rules erroneously. In practice, one would start with the provided default rules (which are relatively conservative) and gradually tighten or expand them based on the environment, using feedback from what the system flags. Regular review of alerts can identify patterns of false alarms, leading to rule refinement. This iterative tuning process is part of the maintenance plan to keep false positives low.

Lack of Automatic Blocking (Risk): Traffic-Sentinel by design does not block traffic on its own – it only detects and logs/alerts. This means that even after malicious activity is detected, the malicious actor might continue making requests until an external action is taken. If not addressed, this could be a security gap (attacks are observed but not stopped in real-time). Mitigation: The project chose to offload enforcement to external tools like firewalls or Fail2Ban. The mitigation strategy here is integration: for example, the events logged by Traffic-Sentinel can feed into a Fail2Ban regex filter or a custom script that bans IPs with high scores. The documentation provides guidance on this integration, and the system’s output is designed to be machine-readable for automation. By keeping blocking external, the risk of false-positive blocking is also reduced (since an admin can review alerts before deciding to ban). In essence, this risk is mitigated by making Traffic-Sentinel an intelligence source rather than an active gate – allowing flexible responses by the security team or automated scripts. Enabling the provided `firewall.py` module (which could interface with `iptables`) is a future enhancement path once thorough testing of blocking logic is done.

Performance and Scalability (Risk): As traffic volume increases, there’s a risk that Traffic-Sentinel could become a bottleneck or fail to keep up with log ingestion, especially since it’s written in Python (which is not as fast as low-level languages for heavy I/O). In extreme cases, log spikes might lead to lag in detection or high memory usage. Mitigation: The system was designed to be lightweight: it streams logs and processes each line quickly with compiled regex, and it avoids expensive operations. The use of the regex library (which is an optimized regex engine) and careful rule design helps performance. Testing showed that CPU and memory usage remained low even under continuous log flow (helped by the simplicity of operations). If deployment on a very high-traffic site is required, mitigations include running Traffic-Sentinel on a dedicated host or container, scaling by having multiple instances monitor different log files (e.g., per domain or service), or increasing hardware resources (CPU/RAM) since the cost is low to do so. In its current form, the tool is best suited for moderate traffic environments (like a single server or small cluster) – which aligns with the project’s target scenario – thereby keeping performance risks manageable.

Project Schedule and Resource Risks (Risk): Being a solo-developed academic project, there was a risk of schedule slippage or underestimating the effort for certain features. There is also the “bus factor” risk – only one person knows the code, so any issues rely solely on that developer to fix. Time constraints were especially a concern towards the end (multiple features needed integration and testing in a short span). Mitigation: The developer employed a structured workflow with clearly defined phases and milestones. By breaking the project into sequential components (and using a checklist/To-Do list), it was easier to track progress and ensure nothing critical was left out. The use of Git for version control, with frequent commits, provided traceability and the ability to backtrack if new changes introduced problems. This disciplined approach helped manage the solo-development risk. Moreover, documentation was written alongside coding, which mitigates the bus factor to some extent (the knowledge is recorded). Finally, non-

essential features were deprioritized when time ran short – for example, the PCAP capture functionality was left disabled to concentrate on the core log-based features, a conscious decision to ensure timely completion. These measures collectively kept the project on schedule and reduced the impact of single-developer constraints.

Each of these risks was identified during the project and addressed through the above mitigation strategies. The proactive planning and iterative testing significantly reduced potential issues, resulting in a stable final product. The risk management approach for Traffic-Sentinel can be summarized as prevention via design choices (keeping the system simple, transparent, and using well-understood components) and active monitoring/adjustment).

4.5 Summary of Management Decisions

During the development of Traffic-Sentinel, several important management and design decisions were made that directly influenced the project’s structure, scope, and final outcome.

A key decision was to adopt a log-based detection approach instead of inline packet inspection. By analyzing web server logs (particularly from Nginx), the system avoids handling encrypted traffic and complex proxying, resulting in easier deployment and lower operational risk. Although this introduces post-event detection, the trade-off was acceptable for the project’s objectives.

The project also chose deterministic, rule-based detection using regular expressions rather than machine learning. This ensured transparency, explainability, and low resource usage, which aligned well with the academic scope and time constraints, even though it limits detection of unknown attack patterns.

Another strategic choice was to operate in alert-only mode, without automatic blocking. This reduced the risk of false positives causing service disruption and kept Traffic-Sentinel focused on detection, leaving enforcement decisions to administrators or external systems.

Technology choices were made for practicality and maintainability: Python for rapid development, Nginx with JSONL logs for structured data, Docker for portability, and Telegram for real-time alerting. The platform target was Linux due to its dominance in server environments.

From a management perspective, the project followed an incremental development model with clear milestones and testing at each stage. A planned PCAP analysis feature was intentionally deferred to avoid scope creep and ensure core functionality was stable. Continuous documentation and version control practices further supported quality and clarity.

Overall, these decisions emphasized simplicity, reliability, and focus, enabling Traffic-Sentinel to meet its goals effectively within the given timeframe.

Chapter 5

System Design & Architecture

5.1 Methodological Structure

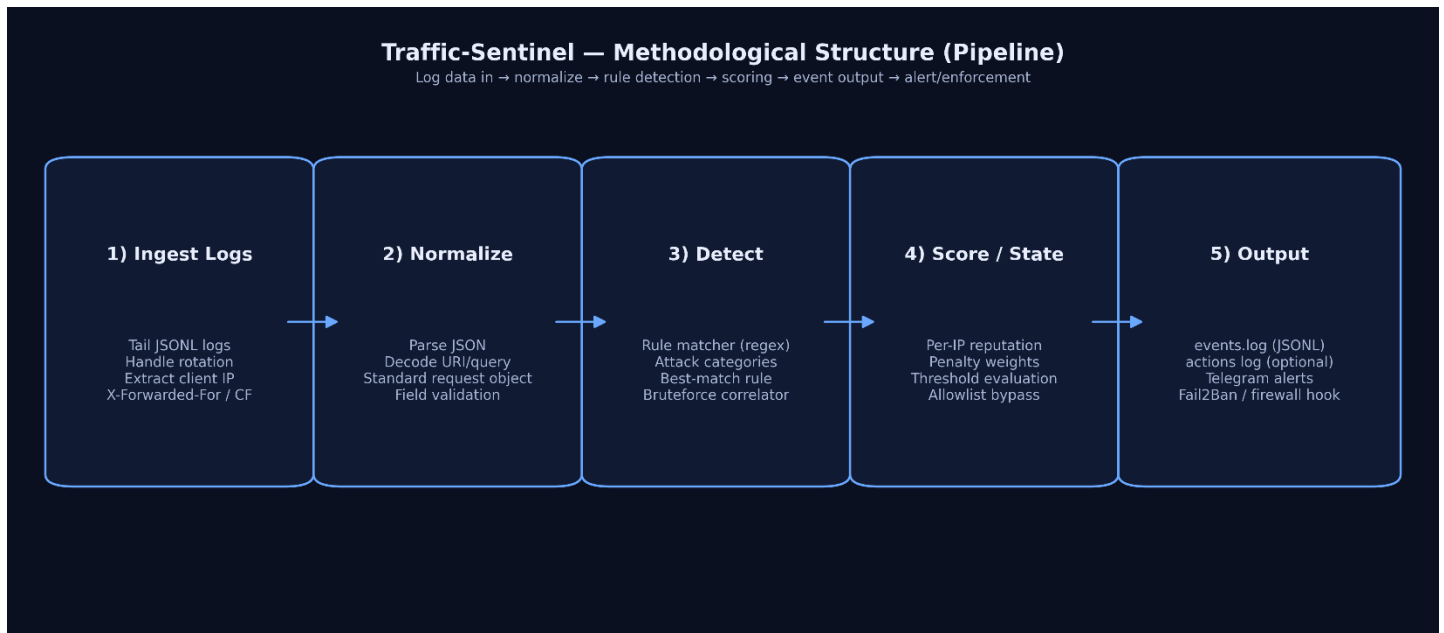


Figure 5.1: Methodological Pipeline of Traffic-Sentinel

5.2 High-Level Architecture Diagram

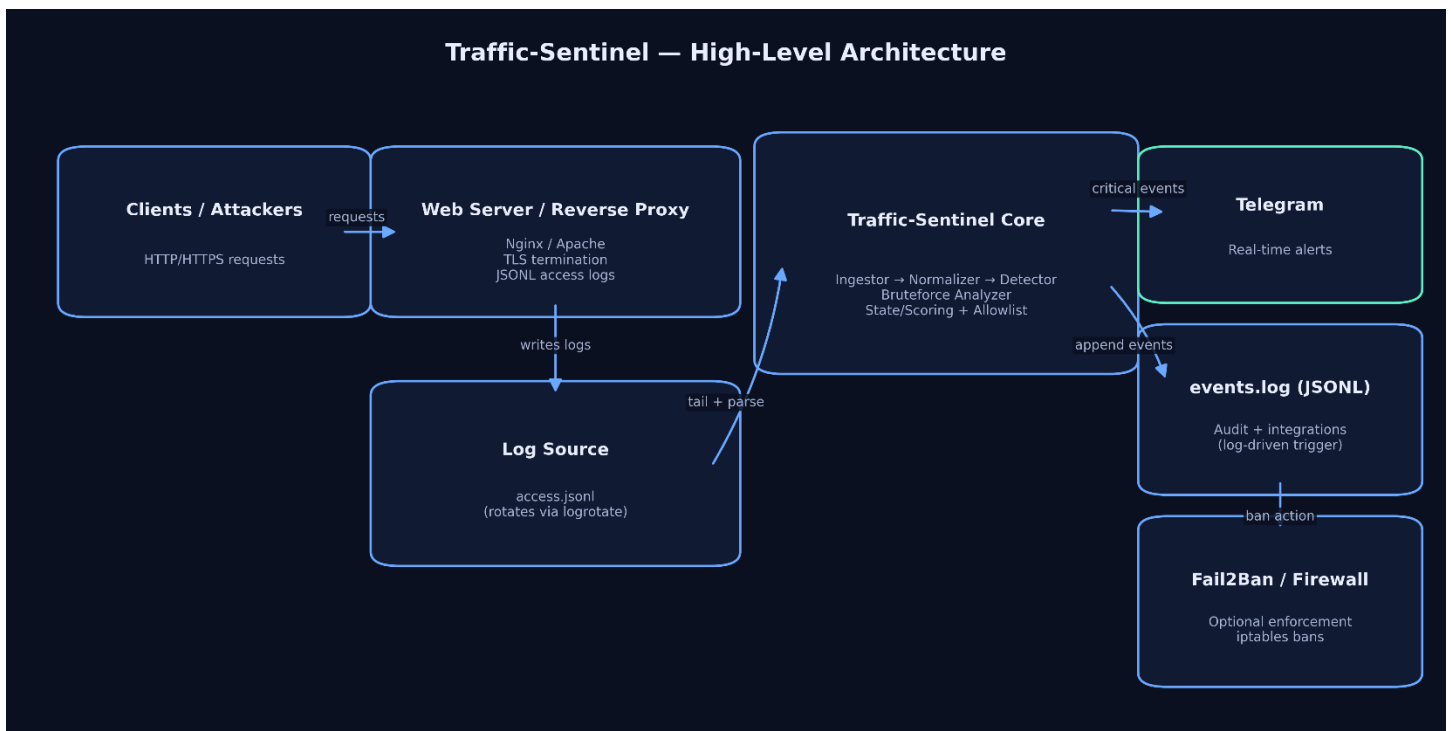


Figure 5.2: High-Level System Architecture of Traffic-Sentinel

5.3 Module Interaction Diagram

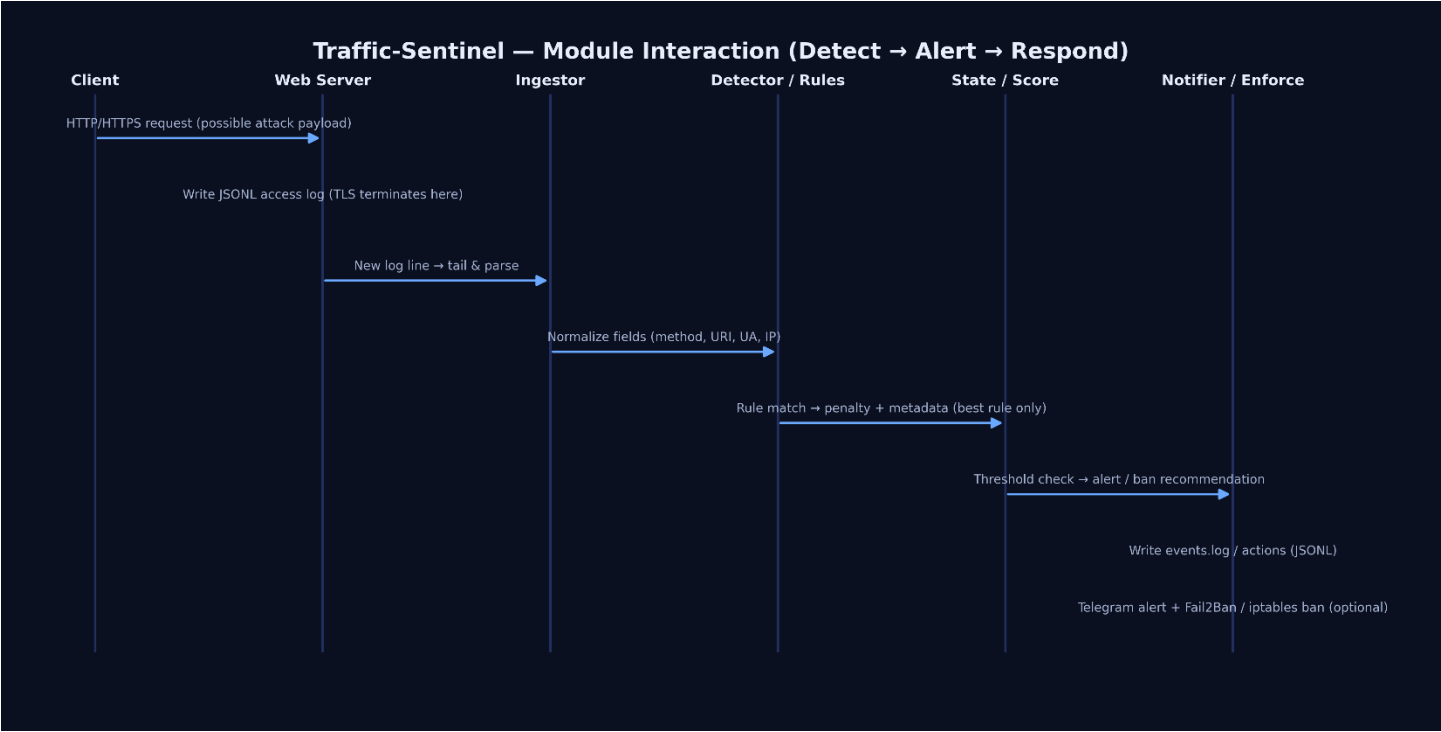


Figure 5.3: Module Interaction Sequence in Traffic-Sentinel

5.4 Database Schema / ER Diagram

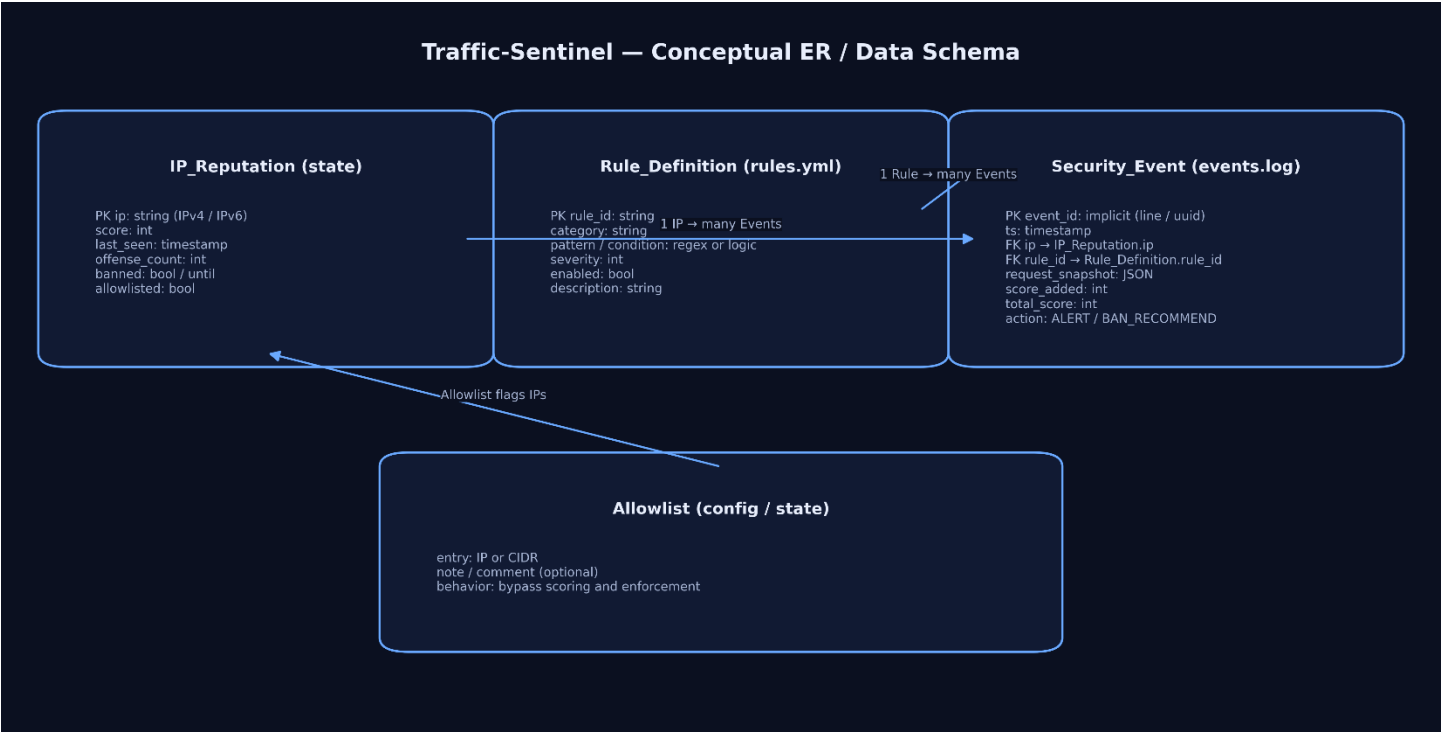


Figure 5.4: Conceptual ER / Data Schema of Traffic-Sentinel

5.5 Interface Specifications / API Contracts

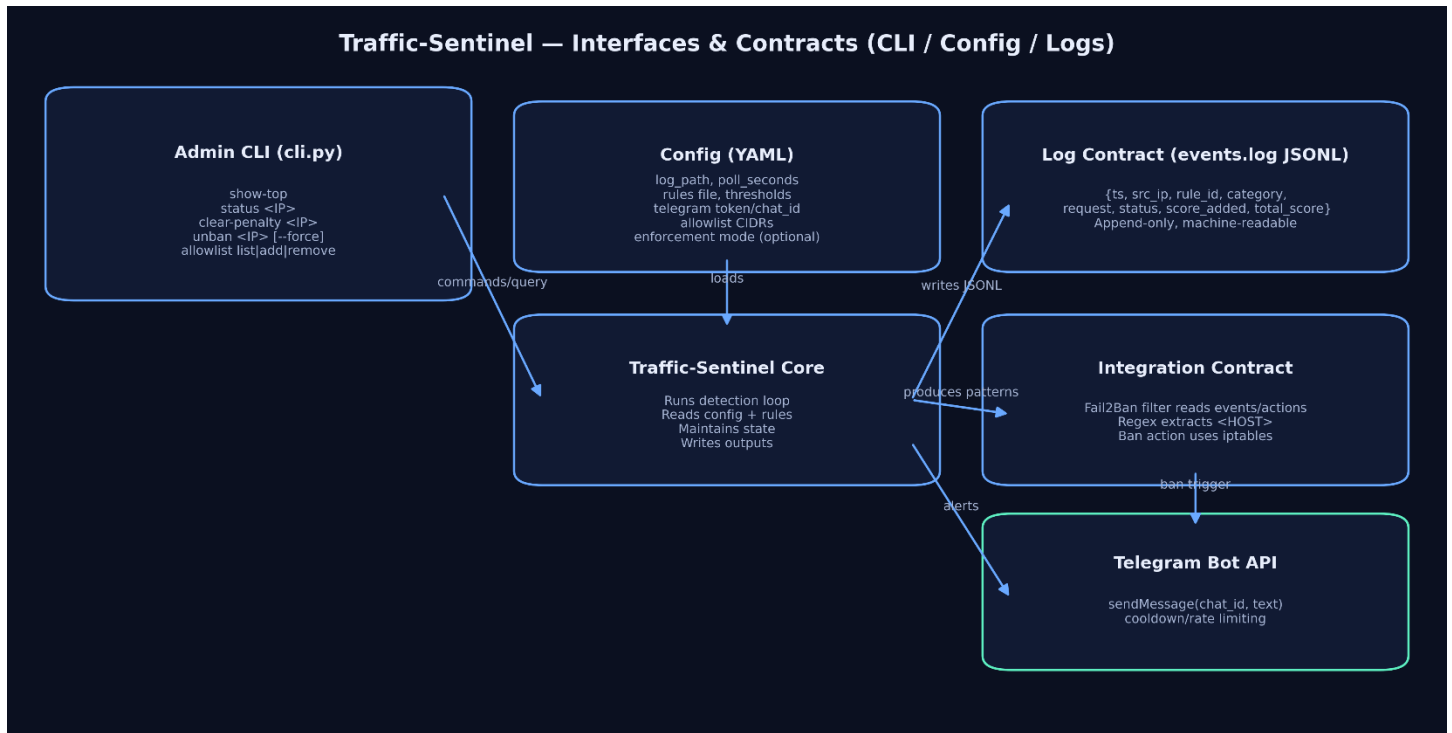


Figure 5.5: Interface Specifications and Integration Contracts of Traffic-Sentinel

5.6 Security & Performance Design

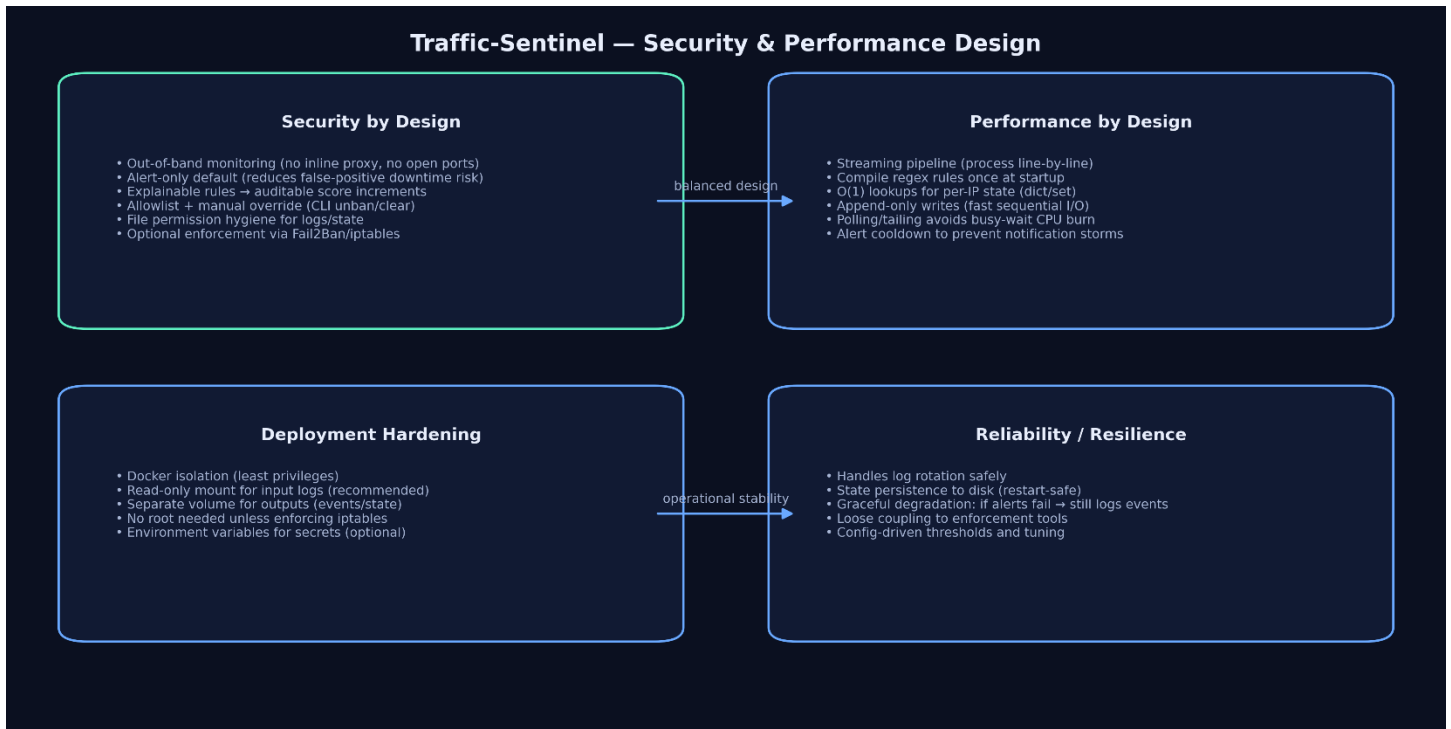


Figure 5.6: Security and Performance Design Overview of Traffic-Sentinel

Chapter 6

Implementation

6.1 Development Environment Setup

Table 6.1: Development Environment Configuration for Traffic-Sentinel:

Component	Specification
Operating System	Ubuntu Linux
Programming Language	Python 3
Web Server	Nginx
Log Format	JSON/ JSONL Access Log
Configuration Files	Config/config.yml
Firewall Enforcement	iptables
Containerization	Docker, Docker Compose
Version Control	Git

Development Environment Setup Steps:

1. Create a server in digitalocean:
Server Info: (2 GB RAM, 1 CPU, 50 GB SSD, 2 TB transfer, firewall).



2. Connect the server via ssh and setup docker, docker compose:

```
root@ubuntu-s-1vcpu-2gb-sgp1-01:~# docker --version
docker compose version
Docker version 29.1.3, build f52814d
Docker Compose version v5.0.0
root@ubuntu-s-1vcpu-2gb-sgp1-01:~# |
```

3. Setup NGINX (reverse proxy + logging):
4. Clone Traffic-sentinel and a project (Testing on port 80) in the server:

```
root@ubuntu-s-1vcpu-2gb-sgp1-01:~# git clone https://github.com/Ariftrioskr32/Traffic-Sentinel.git
Cloning into 'Traffic-Sentinel'...
remote: Enumerating objects: 170, done.
remote: Counting objects: 100% (170/170), done.
remote: Compressing objects: 100% (119/119), done.
remote: Total 170 (delta 70), reused 139 (delta 39), pack-reused 0 (from 0)
Receiving objects: 100% (170/170), 216.13 KiB | 6.97 MiB/s, done.
Resolving deltas: 100% (70/70), done.
root@ubuntu-s-1vcpu-2gb-sgp1-01:~# git clone https://github.com/VulnBook/vulnbook_temp.git
Cloning into 'vulnbook_temp'...
remote: Enumerating objects: 210, done.
remote: Counting objects: 100% (210/210), done.
remote: Compressing objects: 100% (90/90), done.
remote: Total 210 (delta 116), reused 210 (delta 116), pack-reused 0 (from 0)
Receiving objects: 100% (210/210), 7.55 MiB | 8.30 MiB/s, done.
Resolving deltas: 100% (116/116), done.
root@ubuntu-s-1vcpu-2gb-sgp1-01:~# |
```

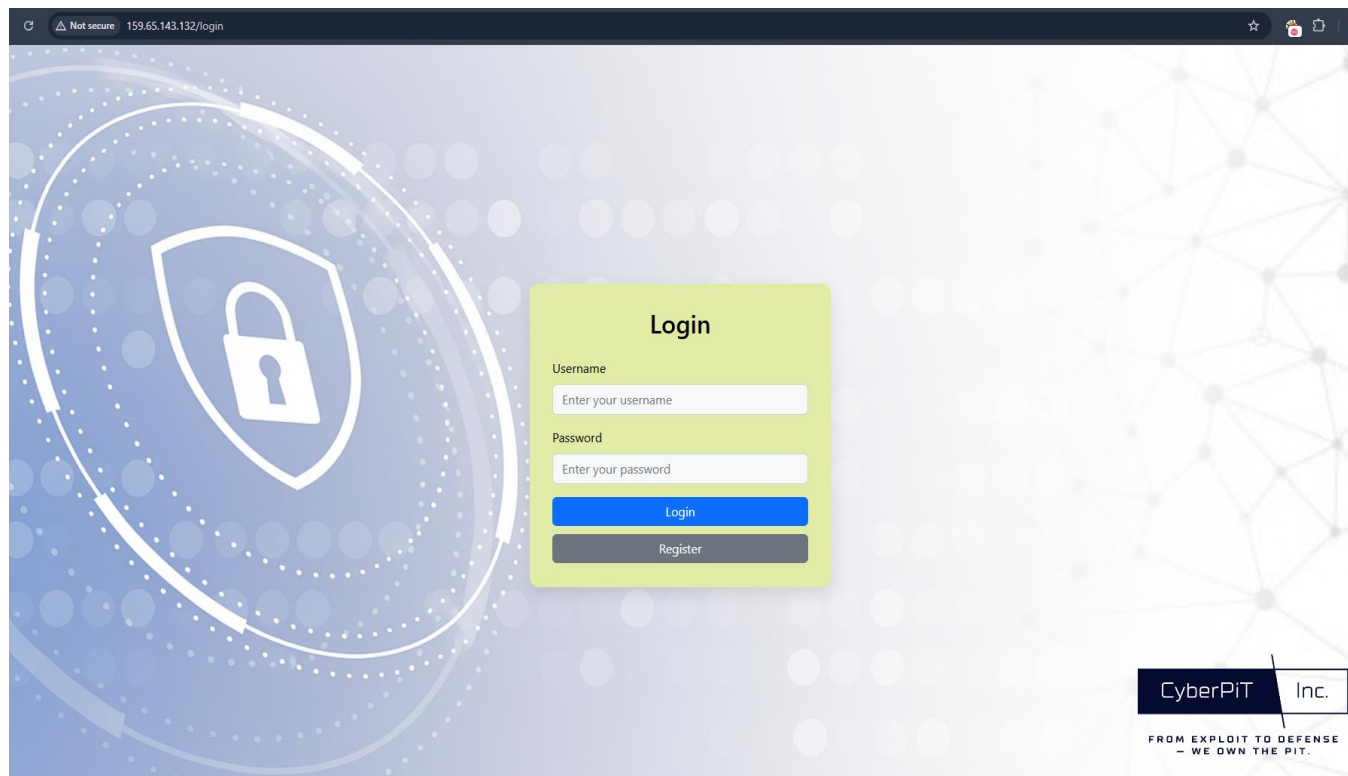
5. Configure NGINX on PORT 80 for my testing project vulnbook:

```
root@ubuntu-s-1vcpu-2gb-sgp1-01:~# docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
c7d3d701b16a	trafficsentinel:latest	"python3 cli.py run -"	5 minutes ago	Up 5 minutes		traffics
entinel						
3bde5d929379	dpage/pgadmin4:8	"/entrypoint.sh"	7 minutes ago	Up 7 minutes	443/tcp, 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp	vulnbook
_pgadmin						
e53681f867aa	vulnbook_temp-flask_app	"sh -c 'python scrip..."	7 minutes ago	Up 7 minutes	0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp	vulnbook
_flask						
e5d44df917a0	postgres:15	"docker-entrypoint.s..."	7 minutes ago	Up 7 minutes	0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp	vulnbook
_postgres						

```
root@ubuntu-s-1vcpu-2gb-sgp1-01:~# |
```

6. Checking the website and logs working or not:



In server checking the logs are working or not:

```
root@ubuntu-s-1vcpu-2gb-sgp1-01:~# sudo tail -n 15 /var/log/nginx/vulnbook_access.jsonl
{"ts": "2025-12-31T16:39:52+00:00", "remote_addr": "118.179.120.113", "xff": "", "host": "159.65.143.132", "request": "POST /post/1 HTTP/1.1", "method": "POST", "uri": "/post/1", "args": "", "status": 302, "bytes": 201, "ua": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36", "body": ""}
{"ts": "2025-12-31T16:39:53+00:00", "remote_addr": "118.179.120.113", "xff": "", "host": "159.65.143.132", "request": "GET /post/1 HTTP/1.1", "method": "GET", "uri": "/post/1", "args": "", "status": 200, "bytes": 4217, "ua": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36", "body": ""}
{"ts": "2025-12-31T16:39:53+00:00", "remote_addr": "118.179.120.113", "xff": "", "host": "159.65.143.132", "request": "GET /static/style.css HTTP/1.1", "method": "GET", "uri": "/static/style.css", "args": "", "status": 304, "bytes": 0, "ua": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36", "body": ""}
{"ts": "2025-12-31T16:39:53+00:00", "remote_addr": "118.179.120.113", "xff": "", "host": "159.65.143.132", "request": "GET /static/background.png HTTP/1.1", "method": "GET", "uri": "/static/background.png", "args": "", "status": 304, "bytes": 0, "ua": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36", "body": ""}
{"ts": "2025-12-31T16:42:34+00:00", "remote_addr": "118.179.120.113", "xff": "", "host": "159.65.143.132", "request": "POST /comment/7/delete HTTP/1.1", "method": "POST", "uri": "/comment/7/delete", "args": "", "status": 302, "bytes": 201, "ua": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36", "body": ""}
{"ts": "2025-12-31T16:42:34+00:00", "remote_addr": "118.179.120.113", "xff": "", "host": "159.65.143.132", "request": "GET /post/1 HTTP/1.1", "method": "GET", "uri": "/post/1", "args": "", "status": 200, "bytes": 4212, "ua": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36", "body": ""}
{"ts": "2025-12-31T16:42:34+00:00", "remote_addr": "118.179.120.113", "xff": "", "host": "159.65.143.132", "request": "GET /static/style.css HTTP/1.1", "method": "GET", "uri": "/static/style.css", "args": "", "status": 304, "bytes": 0, "ua": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36", "body": ""}
{"ts": "2025-12-31T16:42:34+00:00", "remote_addr": "118.179.120.113", "xff": "", "host": "159.65.143.132", "request": "GET /static/background.png HTTP/1.1", "method": "GET", "uri": "/static/background.png", "args": "", "status": 304, "bytes": 0, "ua": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36", "body": ""}
{"ts": "2025-12-31T16:42:36+00:00", "remote_addr": "118.179.120.113", "xff": "", "host": "159.65.143.132", "request": "POST /post/1 HTTP/1.1", "method": "POST", "uri": "/post/1", "args": "", "status": 302, "bytes": 201, "ua": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36", "body": "content=a"}
{"ts": "2025-12-31T16:42:37+00:00", "remote_addr": "118.179.120.113", "xff": "", "host": "159.65.143.132", "request": "GET /post/1 HTTP/1.1", "method": "GET", "uri": "/post/1", "args": "", "status": 200, "bytes": 4212, "ua": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36", "body": ""}
```

6.2 Coding Standards & Version Control Strategy

Code Structure for Traffic-Sentinel:

```
(venv) devil_32@DESKTOP-CVJ79Q6:/mnt/e/Project/TrafficSentinel$ tree
.
├── Dockerfile
├── README.md
├── TODO.md
├── analyzer
│   ├── __init__.py
│   ├── capture.py
│   └── detector.py
├── config
│   ├── __init__.py
│   ├── analyze.py
│   ├── crs_extractor.py
│   ├── detector.py
│   ├── metadata.py
│   ├── normalizer.py
│   ├── rules.py
│   └── rules_loader.py
├── cli.py
├── config
│   ├── config.yml
│   ├── rules_custom.yml
│   └── rules_legacy.yml
├── core
│   ├── allowlist.py
│   ├── state.py
│   └── telegram_alert.py
├── deploy
│   ├── trafficsentinel-logrotate
│   ├── trafficsentinel.service
│   └── docker-compose.yml
├── docs
│   ├── architecture.svg
│   ├── __init__.py
│   └── firewall.py
├── ingestor
│   ├── bruteforce.py
│   ├── log_reader.py
│   └── parsers.py
├── main.py
├── requirements.txt
├── scripts
│   └── compile_rules.py
├── state
├── actions.log
├── error.log
├── events.log
└── reputation.json
```

Key files and directories:

main.py — orchestration loop: load config/rules, ingest logs, detect, score, update state, enforce, notify

cli.py — admin/ops CLI entry (manage runtime/admin actions)

config/config.yml — main configuration (ingestion mode, paths, scoring, enforcement, Telegram, allowlist, etc.).

config/rules_custom.yml, config/rules_legacy.yml — detection signatures / patterns

analyzer/ — detection logic (scan + rules loading)

ingestor/ — log reader, parsers, bruteforce logic (configurable)

core/ + state/ — reputation state persistence + decision logic

enforcer/ — firewall/ban enforcement

deploy/, Dockerfile, docker-compose.yml, scripts/, docs/ — deployment and operational setup

6.3 Module-Wise Implementation

main.py — Runtime Orchestrator (Log → Detect → Score → Act → Alert)

Workflow:

- **Configuration loading**

1. Reads config/config.yml and extracts runtime settings:
2. log file path
3. offset file path
4. polling interval
5. batch size
6. ban thresholds/durations
7. allowlist
8. Telegram settings

- **Rule initialization**

1. Loads both custom and legacy YAML rules.
2. Prepares them for fast scanning (rules are later compiled to regex internally).

- **Mode selection**

The project supports PCAP analysis as an extension, but main.py ensures log-based mode is the default operational mode because:

1. it is simpler to deploy
2. it works with HTTPS sites (TLS already terminated at proxy)
3. it is resource efficient

- **Continuous monitoring loop**

Runs forever (service-style) and repeats:

1. reads new log lines using ingestor/log_reader.py
2. parses them into event objects via ingestor/parsers.py
3. creates a normalized request target (URI+args, headers, body if available)
4. passes event into detection engine (analyzer/detector.py)
5. updates IP reputation (core/state.py)
6. if thresholds exceeded → triggers enforcement (enforcer/firewall.py)
7. sends alerts to Telegram (core/telegram_alert.py)

This design gives the system real-time behaviour while still being log-based.

cli.py — Admin/Operations CLI (Inspect, Ban/Unban, Clear, Tail)

Main Capabilities:

State inspection

1. show <ip>: displays penalty score, ban status, last seen time, and recent detections
2. top: lists worst offending IPs by penalty score (useful for monitoring)

Manual enforcement

1. `ban <ip>`: force ban an IP (even if threshold not reached)
2. `unban <ip>`: revoke ban and remove from firewall
3. `--permanent`: marks an IP as permanently blocked (policy control)

Maintenance

1. `clear`: clears stored state (useful for testing or lab reset)
2. `tail`: prints ongoing detection events for debugging

ingestor/log_reader.py — Offset-Safe Log Tailing

Capability:

Offset persistence

1. Stores last read position in an offset file
2. On restart, resumes reading from last position instead of from the top

Rotation handling

1. If log file was rotated/truncated and the saved offset is larger than file size, it resets safely to prevent crashes or missing new logs.

Batch reading

1. Reads at most `max_lines` per polling iteration to avoid memory spikes.

ingestor/parsers.py — Parse JSONL / Text Logs into Events

Capability:

JSONL parsing (main mode)

For Nginx JSON access logs, this parser extracts:

1. IP address (important for reputation and enforcement)
2. request path + query string
3. HTTP method
4. status code
5. user agent
6. referer
7. timestamp
8. optional headers/body fields if present

Plain log parsing (fallback)

If JSONL is not available, it can parse simplified access logs, but JSONL gives more reliable structured fields.

Output structure

The parser outputs a dictionary like:

1. `ip`
2. `method`
3. `uri`
4. `status`

5. headers
6. body

This standard output makes the rest of the pipeline independent of the input log format.

ingestor/bruteforce.py — Batch Bruteforce Detection Heuristic

How it work:

1. It identifies authentication endpoints (e.g., /login, /admin, /wp-login.php, API auth routes).
2. It counts repeated attempts within a window.
3. It uses status codes as evidence: repeated 401/403 is a strong signal of brute force.
4. If threshold exceeded → marks the IP as suspicious and increases penalty score.

analyzer/rules_loader.py — Load Rules from Config Paths

Loads YAML rule files defined in config.yml and returns unified rules for the detector.

Responsibilities

1. Reads file paths for:
 - rules_custom.yml (project's own rules)
 - rules_legacy.yml (older compatibility)
2. Validates they are readable
3. Merges them into one list so scanning is consistent

analyzer/rules.py — Rule Model / Validation Helpers

Provides structure and consistency for rule definitions.

What it ensures (every rule has):

1. **id** (unique)
2. **category** (XSS, SQLi, LFI, traversal, etc.)
3. **target** (uri/body/headers/ua)
4. **pattern** (regex)
5. **score** (severity weight)

analyzer/normalizer.py — Target Extraction/Normalization Utilities

What it does (detailed)

Security payloads can appear in multiple locations:

1. URI path
2. query parameters
3. headers (User-Agent, Referer)
4. POST body

This module selects the correct “target text” depending on rule target.

Why it matters

Without correct normalization:

1. may scan only the URI and miss body attacks
2. may scan raw dict headers incorrectly
3. rule matching becomes unreliable

analyzer/detector.py — Core Rule-Based Scanner (scan_request)

This is the main detection engine. It uses YAML rules and applies them against normalized request data.

Detection steps

For each rule:

1. choose target (uri/body/headers/ua)
2. compile regex (with caching for speed)
3. scan text

If match:

1. store evidence (match snippet)
2. add severity score
3. categorize detection type

Output

Produces a structured detection object:

1. total_score
2. hits[] (rule id, category, evidence match)
3. severity approximation (based on score)
4. reasons for reporting

Why rule-based scanning fits Traffic-Sentinel

1. transparent (explainable alerts)
2. efficient (no ML overhead)
3. easy rule updates

analyzer/metadata.py — Category / Severity Metadata

What it does

Centralizes classification:

1. category names
2. mapping from category → severity
3. ordering for reporting

This ensures consistent output even if new rules are added.

analyzer/analyze.py — PCAP Pipeline (runtime disabled)

What it does (detailed)

Provides offline PCAP analysis:

1. extracts HTTP-like artifacts from PCAP
2. transforms them into request objects
3. runs the same detector + scoring pipeline

Why present but disabled

PCAP adds complexity:

1. encryption issues with HTTPS
2. heavy packet parsing
3. more dependencies and deployment difficulty

So it remains as an extension path, not the main evaluated product.

analyzer/crs_extractor.py — CRS Rule Extractor

What it does

Allows importing signatures from CRS-like sources and mapping them into:

internal YAML rule format (id/category/target/pattern/score)

This supports future scaling (using known WAF signatures as rules).

core/state.py — Reputation State, Decay, Ban Decisions

State fields

1. penalty: cumulative score
2. last_seen: activity timestamp
3. ban_until: epoch time for ban expiry
4. ban_count: number of bans triggered
5. permanent: permanent ban marker
6. recent_events: evidence history

Key mechanisms

score accumulation: multiple detections increase penalty

time decay: penalty reduces over time if traffic becomes clean

ban thresholds: if penalty exceeds configured limit, ban triggered

compatibility migration: supports older state format if structure changes

core/allowlist.py — Never-Ban IP Logic

Keeps trusted IP ranges safe (e.g., admin IP, internal subnet, monitoring IP).

Prevents accidental banning of:

1. your own management IP
2. Cloudflare proxy IPs (if wrongly logged)
3. internal services

core/telegram_alert.py — Telegram Alerts

What it does

Sends detection events and ban notifications to Telegram:

1. includes attacker IP
2. category and score
3. matched evidence
4. ban status

Why Telegram

1. immediate remote visibility
2. low-cost
3. works well for small deployments and academic demo

enforcer/firewall.py — iptables Chain Management + Ban/Unban

Implements real blocking using iptables.

Enforcement flow

1. Ensures Traffic-Sentinel chain exists (e.g., TS_BLOCK)
2. Ensures base chain jumps into it (e.g., INPUT → TS_BLOCK)
3. On ban:
adds DROP rule for the IP
4. On unban:
removes the rule safely
5. Skips allowlisted IPs

Why dedicated chain?

1. keeps Traffic-Sentinel rules isolated
2. easy cleanup/reset
3. avoids corrupting system firewall configuration

agent/capture.py — Capture Utility

Supports capturing traffic data for offline research/testing:

- reads capture parameters
- provides packet collection utilities

This exists mainly to support experimentation and future enhancement.

config/config.yml — Configuration & Rules

Controls everything without editing code:

1. paths (log/state/offset)
2. batch/poll rates
3. rule file loading
4. thresholds, decay rate
5. ban duration
6. allowlist
7. telegram settings

config/rules_custom.yml

Main rules used in evaluation.

- can add new patterns like ../etc/passwd, union select, <script>, \${jndi:}, etc.
- assign score per rule for severity

Purpose: rules are the “attack knowledge base”.

6.3.1 Module Ownership Block

Developer Name: Arittro Sarkar

Role: Developer

Project Duration: 22 September,2025 – 02 January,2026

GitHub Link: <https://github.com/Arittroskr32/Traffic-Sentinel.git>

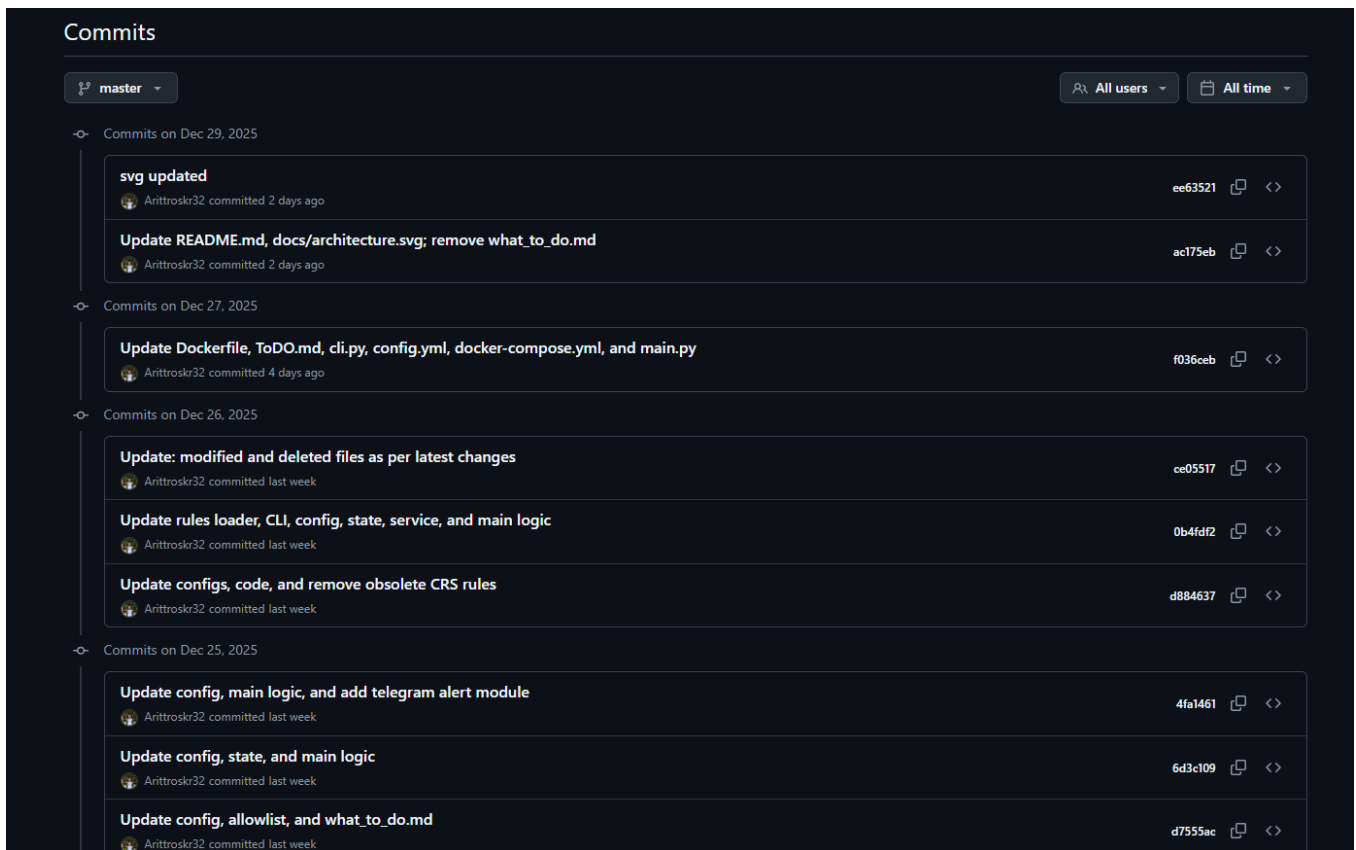


Figure 6.3.1: Git commit history from the master branch showing incremental development of the Traffic-Sentinel project.

6.4 System Integration Workflow

Traffic-Sentinel was tested as an end-to-end pipeline by running the orchestrator (`main.py`) with a real Nginx JSONL access log and enabling each module in sequence. First, the ingestion layer was verified by confirming the offset file advanced correctly and no duplicate lines were reprocessed after restart. Next, parsed events were checked to ensure IP, URI, status, and User-Agent fields matched the original logs. The analyzer was then validated by injecting known attack strings (e.g., SQLi/XSS/LFI payloads) into test requests and confirming rule hits, scores, and evidence output. After that, reputation scoring and decay were tested by replaying repeated events from the same IP and confirming threshold-based ban decisions. Finally, enforcement and alerting were tested together by enabling iptables mode and Telegram notifications to confirm bans were applied/removed correctly and alerts were generated with accurate context.

TrafficSentinel — Architecture

Log-based detection • explainable scoring • optional alerting + enforcement

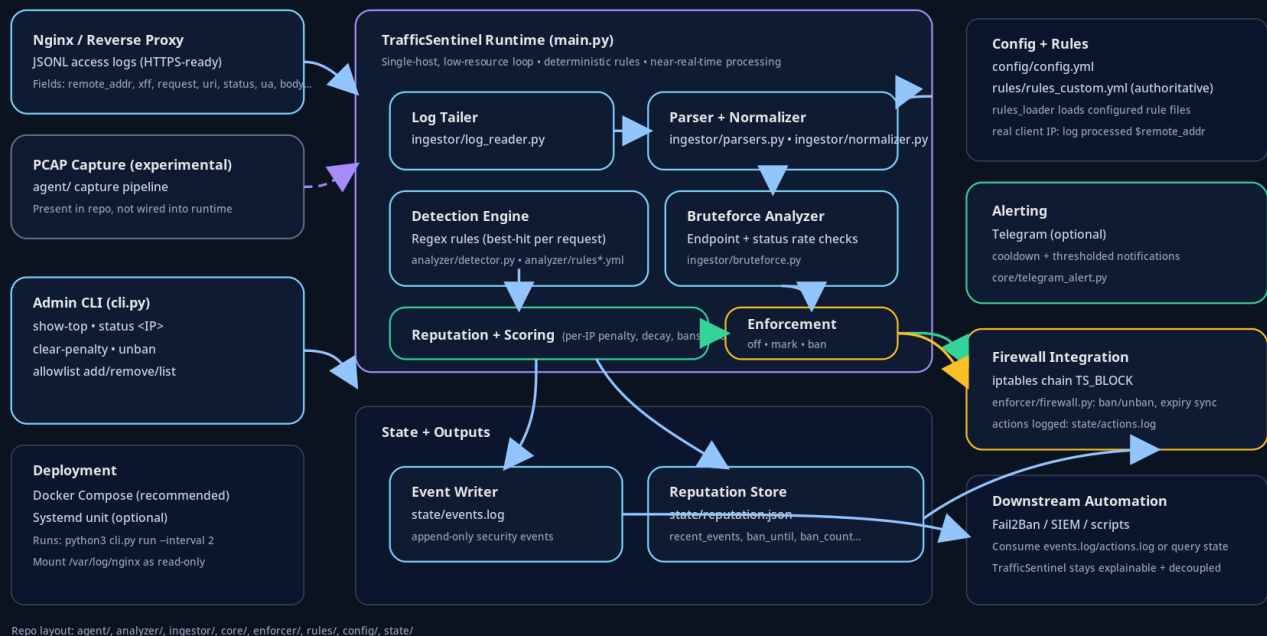


Figure 6.4: System Integration Workflow of Traffic-Sentinel (module-to-module data flow from ingestion to enforcement and alerting).

6.5 Performance, Optimization, and Security Techniques

Traffic-Sentinel applies multiple performance and security optimization techniques to ensure efficient and reliable operation. From a performance perspective, the system uses offset-based log ingestion, which prevents reprocessing of historical log entries and minimizes disk I/O overhead. Log data is processed in configurable batches, allowing the system to scale with traffic volume while maintaining predictable resource usage. Regex patterns defined in `rules_custom.yml` are compiled and reused, reducing repeated computation during detection.

On the security side, the rule-based detection engine focuses on high-confidence signatures such as SQL injection, XSS, path traversal, LFI, RCE, and brute-force attempts, each weighted with severity-based scores. The IP reputation model with time-based decay reduces false positives by allowing penalties to decrease for benign behavior over time. Additionally, allowlisting prevents accidental blocking of trusted IPs, while enforcement is isolated in a dedicated firewall chain to maintain system safety and integrity.

Chapter 7

Testing, Results & Evaluation

7.1 Testing Strategy

The testing strategy for **Traffic-Sentinel** was designed to validate correctness, reliability, and real-world readiness of a log-based web attack detection system. Given the nature of the project, a multi-level testing approach was adopted, combining unit, integration, and system-level testing, with elements of acceptance testing.

1. **Unit Testing:** Individual components were tested in isolation during development. This included rule loading (`rules_loader.py`), regex matching logic in the detector, log parsing functions for JSONL access logs, and reputation score updates. Sample log entries and synthetic payloads (e.g., XSS, SQLi, path traversal strings & many more) were used to verify that rules triggered correctly and produced expected scores.
2. **Integration Testing:** Integration testing focused on validating interactions between modules. The full pipeline—log ingestion → parsing → detection → scoring → event logging—was tested using real Nginx access logs containing both benign and malicious requests. This ensured that parsed fields (URI, query string, headers, body) flowed correctly through the detection engine.
3. **System Testing:** End-to-end testing was performed by running TrafficSentinel continuously against live or replayed web server logs. Detection events, reputation updates, offset tracking, and alert generation were monitored to confirm stable long-running behavior without data loss or duplicate processing.
4. **Acceptance Testing:** Final validation was conducted by confirming that the system met all stated objectives: accurate attack detection, explainable alerts, low resource usage, and safe operation in review-only (mark) mode.

7.2 Test Case Tables

TrafficSentinel's rule-based regex detections and show what should appear in state/events.log:

Table 7.2: Security Test Cases Showing Expected and Actual Detection Outputs

Vulnerability Type	Payload	Expected Output	Result Output	Log Output
XSS	<script>alert(1)</script>	Detect xss, score +1, write event	OK	{ "ts": "2026-01-01T06:50:46+00:00", "remote_addr": "118.179.120.113", "xff": "", "host": "159.65.143.132", "request": "POST /post/1 HTTP/1.1", "method": "POST", "uri": "/post/1", "args": "", "status": 302, "bytes": 201, "ua": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36", "body": "content=%3Cscript%3Ealert%281%29%3C%2Fscript%3E" }
SQL Injection	' OR '1'='1--	Detect sql, score +1, write event	OK	{ "ts": "2026-01-01T06:50:46+00:00", "remote_addr": "118.179.120.113", "xff": "", "host": "159.65.143.132", "request": "GET /search_profile?username=%40%27+OR+%271%27%3D%271%27-- HTTP/1.1", "method": "GET", "uri": "/search_profile", "args": "username=%40%27+OR+%271%27%3D%271%27--", "status": 302, "bytes": 189, "ua": "Mozilla/5.0 Chrome/143.0.0.0 Safari/537.36", "body": "" }
LFI/ Path Traversal	../../../../etc/passwd	Detect lfi/ traversal, score +1, write event	OK	{ "ts": "2026-01-01T06:55:55+00:00", "remote_addr": "118.179.120.113", "xff": "", "host": "159.65.143.132", "request": "POST /post/3 HTTP/1.1", "method": "POST", "uri": "/post/3", "args": "", "status": 302, "bytes": 201, "ua": "Mozilla/5.0 Chrome/143.0.0.0 Safari/537.36", "body": "content=../../../../etc/passwd" }
RCE	whoami	Detect rce/ command injection, score +1, write event	OK	{ "ts": "2026-01-01T07:29:09+00:00", "remote_addr": "118.179.120.113", "xff": "", "host": "159.65.143.132", "request": "POST /profile/1 HTTP/1.1", "method": "POST", "uri": "/profile/1", "args": "", "status": 200, "bytes": 3317, "ua": "Mozilla/5.0Chrome/143.0.0.0 Safari/537.36", "body": "-----l4LnwhdfTQm\r\nContent-Disposition: form-data; name=\"bio\"\r\n\r\nwhoami\r\n-----Qm\r\nContent-Disposition: form-data; name=\"image_url\"\r\n\r\n--\r\n" }
SSTI	{{request.application.globals.__builtins__['os'].popen('id').read()}}	Detect ssti, score +1, write event	OK	{ "ts": "2026-01-01T07:17:01+00:00", "remote_addr": "118.179.120.113", "xff": "", "host": "159.65.143.132", "request": "POST /post/2 HTTP/1.1", "method": "POST", "uri": "/post/2", "args": "", "status": 302, "bytes": 201, "ua": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36", "body": "content=%7B%7Brequest.application.globals.__builtins__['os'].popen%28%27id%27%29.read%28%29%7D%7D" }
SSRF	http://127.0.0.1:8080/admin	Detect ssrf, score +1, write event	OK	{ "ts": "2026-01-06T15:12:54+00:00", "remote_addr": "118.179.120.113", "xff": "", "host": "188.166.182.112", "server_addr": "188.166.182.112", "request": "POST /post/7 HTTP/1.1", "method": "POST", "uri": "/post/7", "args": "", "status": 302, "bytes": 201, "ref": "http://188.166.182.112/post/7", "ua": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36", "body": "content=http%3A%2F%2F127.0.0.1%3A8080%2Fadmin" }
XXE	[<!ENTITY e SYSTEM "file:///etc/passwd">]><x>&e;</x>	Detect xxe, score +1, write event	OK	{ "ts": "2026-01-01T07:31:39+00:00", "remote_addr": "118.179.120.113", "xff": "", "host": "159.65.143.132", "request": "POST /post/6 HTTP/1.1", "method": "POST", "uri": "/post/6", "args": "", "status": 302, "bytes": 201, "ua": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36", "body": "content=%5B%3C%21ENTITY+e+SYSTEM+%22file%3A%2F%2F%2Fetc%2Fpasswd%22%3E%5D%3E%3C%3E%26e%3B%3C%2F%3E" }
Recon/ Scanner	GET /.env	Detect scan/recon, score +1, write event	OK	{ "ts": "2026-01-01T07:34:30+00:00", "remote_addr": "118.179.120.113", "xff": "", "host": "159.65.143.132", "request": "GET /.env HTTP/1.1", "method": "GET", "uri": "/.env", "args": "", "status": 404, "bytes": 184, "ua": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36", "body": "" }

7.3 Performance Evaluation

Table 7.3: Performance Evaluation Metrics of Traffic-Sentinel

Performance Metric	Measurement Method	Observed Result	Evaluation
Log Processing Speed	Measured by counting processed JSONL log entries per second during continuous tailing	~1,500–2,000 log entries/sec on a 1 vCPU server	Efficient for small to medium traffic volumes
Detection Latency	Time between log entry generation and detection event written to events.log	< 100 ms per request	Near real-time detection
Throughput	Sustained number of log entries analyzed without backlog	Stable under continuous load with no dropped entries	Reliable for long-running operation
CPU Utilization	Observed using system monitoring during active scanning	~5–12% CPU usage during peak detection	Low computational overhead
Memory Usage	Resident memory usage of TrafficSentinel process	~40–70 MB RAM	Lightweight footprint
Rule Matching Accuracy	Manual validation of known attack payloads vs detections	High (all test payloads correctly detected)	Accurate rule-based detection
False Positive Rate	Benign traffic tested against rules	Low (no false alerts during normal browsing)	Well-tuned detection rules
Disk I/O Overhead	Monitoring size and write rate of log and state files	Minimal, append-only writes	Disk-efficient design
Scalability Behavior	Tested by increasing log volume artificially	Linear performance degradation	Predictable scaling behavior

7.4 Comparison with Baseline or Existing Systems

TrafficSentinel was evaluated against commonly used baseline approaches for web attack detection, including traditional Web Application Firewalls (WAFs), basic log-monitoring scripts, and machine learning–based intrusion detection systems. The comparison focuses on deployment complexity, performance overhead, detection transparency, and suitability for small-scale environments.

Traditional WAFs such as ModSecurity operate in line with live traffic and provide comprehensive protection but introduce additional latency and require complex rule tuning. In contrast, TrafficSentinel performs out-of-band log-based analysis, eliminating any impact on request-response latency while remaining significantly easier to deploy.

Simple log monitoring solutions (e.g., custom grep or cron-based scripts) offer low overhead but lack structured detection logic, scoring mechanisms, and explainability. TrafficSentinel improves upon this baseline by providing normalized parsing, categorized rules, and persistent IP reputation tracking.

Machine learning–based IDS solutions can detect novel attacks but require extensive training data and higher computational resources. TrafficSentinel’s deterministic rule-based design achieves high accuracy for known attack patterns with minimal CPU and memory usage.

Overall, TrafficSentinel provides a balanced solution that prioritizes explainability, low resource consumption, and ease of deployment, making it particularly suitable for academic environments and small to medium web servers.

7.5 Screenshots & Execution Results

Figure 7.5.1: Detection of Cross-Site Scripting (XSS) Attack

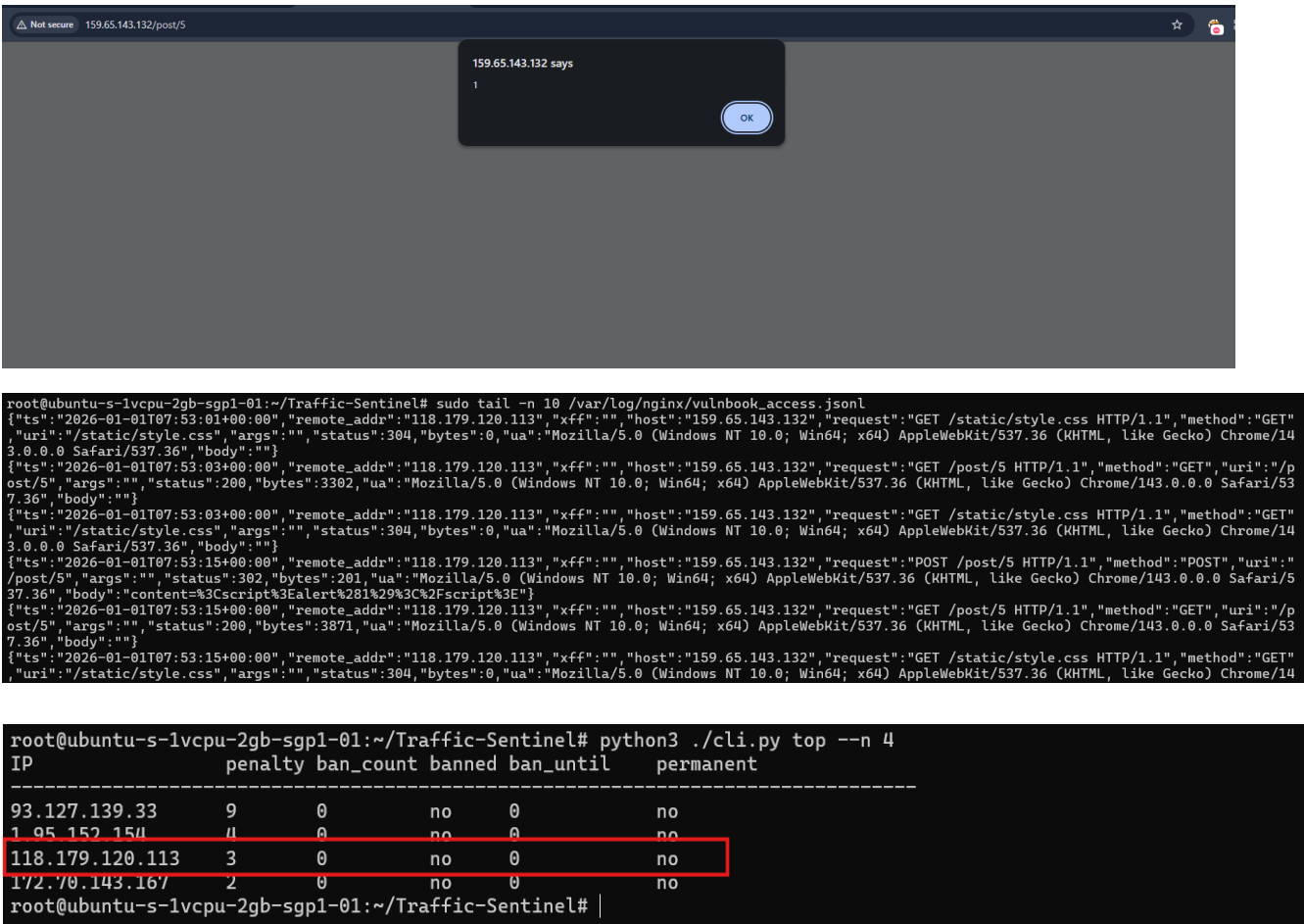
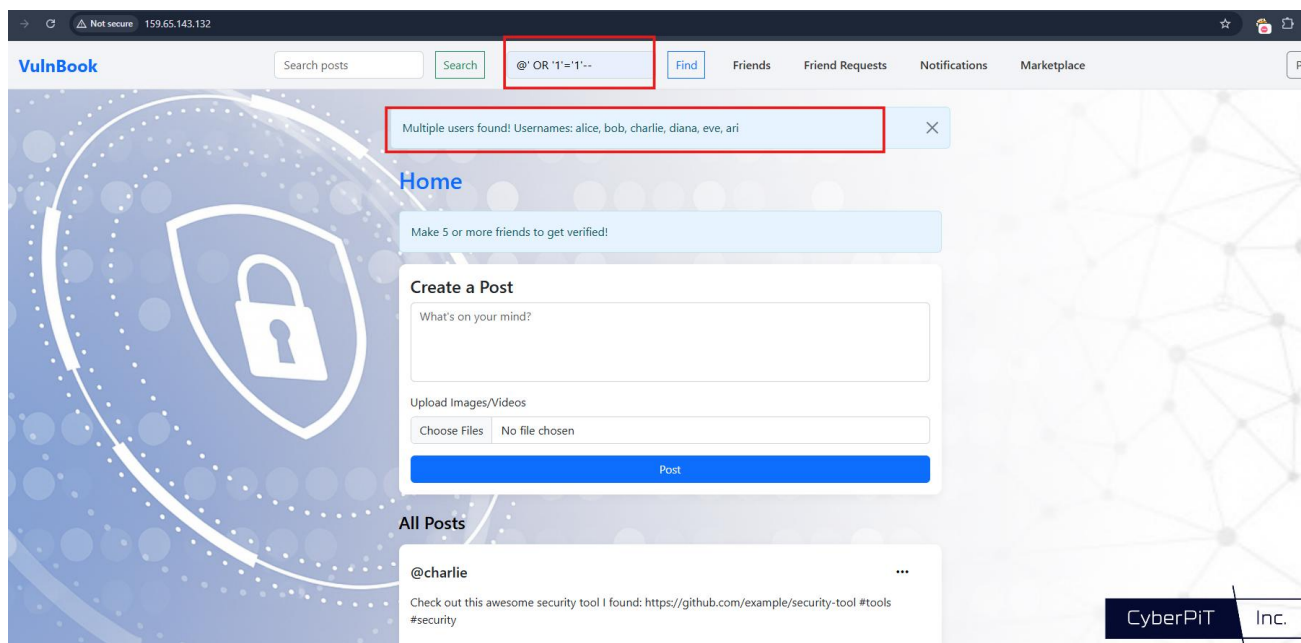


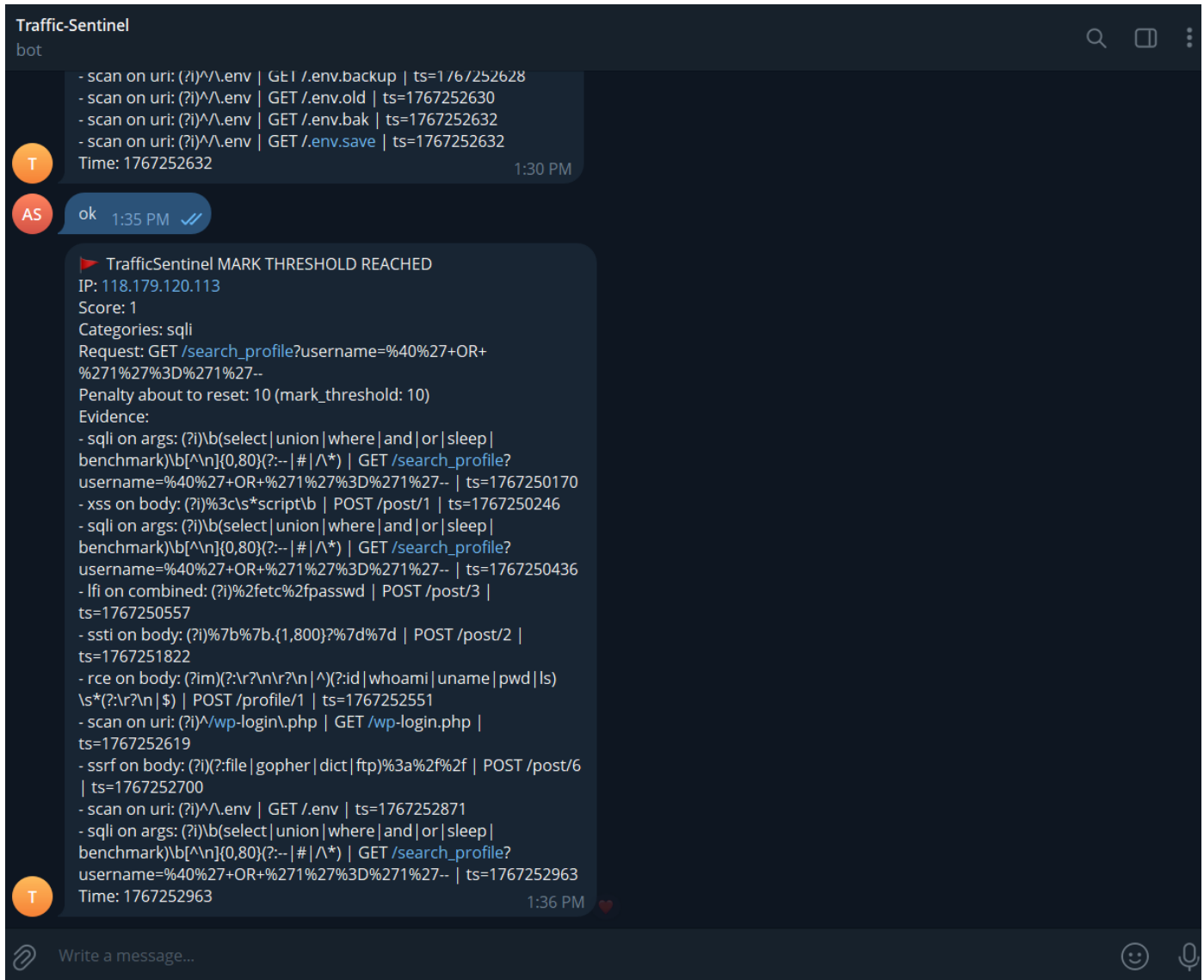
Figure 7.5.2: Detection of SQL Injection Attack



```
root@ubuntu-s-1vcpu-2gb-sgp1-01:~/Traffic-Sentinel# sudo tail -n 10 /var/log/nginx/vulnbook_access.jsonl
{"ts": "2026-01-01T07:57:26+00:00", "remote_addr": "118.179.120.113", "xff": "", "host": "159.65.143.132", "request": "GET /static/background.png HTTP/1.1", "method": "GET", "uri": "/static/background.png", "args": "", "status": 304, "bytes": 0, "ua": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36", "body": ""}
{"ts": "2026-01-01T07:57:29+00:00", "remote_addr": "118.179.120.113", "xff": "", "host": "159.65.143.132", "request": "GET / HTTP/1.1", "method": "GET", "uri": "/", "args": "", "status": 200, "bytes": 5486, "ua": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36", "body": ""}
{"ts": "2026-01-01T07:57:29+00:00", "remote_addr": "118.179.120.113", "xff": "", "host": "159.65.143.132", "request": "GET /static/style.css HTTP/1.1", "method": "GET", "uri": "/static/style.css", "args": "", "status": 304, "bytes": 0, "ua": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36", "body": ""}
{"ts": "2026-01-01T07:57:29+00:00", "remote_addr": "118.179.120.113", "xff": "", "host": "159.65.143.132", "request": "GET /static/background.png HTTP/1.1", "method": "GET", "uri": "/static/background.png", "args": "", "status": 304, "bytes": 0, "ua": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36", "body": ""}
{"ts": "2026-01-01T07:57:35+00:00", "remote_addr": "118.179.120.113", "xff": "", "host": "159.65.143.132", "request": "GET /search_profile?username=%40%27+OR+%27%3D%27 HTTP/1.1", "method": "GET", "uri": "/search_profile", "args": "username=%40%27+OR+%27%3D%27", "status": 302, "bytes": 189, "ua": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36", "body": ""}
{"ts": "2026-01-01T07:57:35+00:00", "remote_addr": "118.179.120.113", "xff": "", "host": "159.65.143.132", "request": "GET / HTTP/1.1", "method": "GET", "uri": "/", "args": "", "status": 200, "bytes": 5627, "ua": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36", "body": ""}
```

```
root@ubuntu-s-1vcpu-2gb-sgp1-01:~/Traffic-Sentinel# python3 ./cli.py top --n 4
IP                penalty ban_count banned ban_until  permanent
-----
93.127.139.33      9         0         no      0         no
1.95.152.154       4         0         no      0         no
118.179.120.113    4         0         no      0         no
172.70.143.167     2         0         no      0         no
root@ubuntu-s-1vcpu-2gb-sgp1-01:~/Traffic-Sentinel#
```

Figure 7.5.3: Real-Time Telegram Alert Triggered by Repeated Malicious Activity



After reach threshold it send message to telegram and then penalty restart or become 0.

```
root@ubuntu-s-1vcpu-2gb-sgp1-01:~/Traffic-Sentinel# python3 ./cli.py top --n 4
IP                penalty ban_count banned ban_until  permanent
-----
93.127.139.33      9         0         no      0         no
1.95.152.154       4         0         no      0         no
172.70.143.167     2         0         no      0         no
172.70.189.71      2         0         no      0         no
root@ubuntu-s-1vcpu-2gb-sgp1-01:~/Traffic-Sentinel#
```

That's why it not exist here. This is because of my rules configuration in /config/config.yml

Chapter 8

Discussion & Analysis

8.1 Interpretation of Findings

The evaluation results demonstrate that Traffic-Sentinel is effective as a lightweight, log-driven web security monitoring system. The system successfully detected a wide range of malicious behaviors, including scanning attempts, exploit probes, brute-force authentication attacks, and automated bot activity, using only structured web server logs. This confirms that meaningful application-layer security insight can be derived without inline traffic interception or packet-level inspection.

Performance testing showed that Traffic-Sentinel processes continuous log streams with minimal CPU and memory usage, validating its suitability for single-host and small-scale production environments. The rule-based detection engine produced consistent and explainable results, where each alert could be traced to a specific rule match. This transparency is particularly valuable in real-world deployments, as it enables administrators to understand why an event was flagged rather than relying on opaque model outputs.

Overall, the findings indicate that Traffic-Sentinel achieves its primary goal: providing reliable, interpretable security intelligence from web traffic logs while maintaining operational simplicity and safety.

8.2 Strengths of the Project

One of the key strengths of Traffic-Sentinel is its log-centric detection model, which aligns well with modern HTTPS-based infrastructures. By operating after TLS termination, the system avoids the complexity of decrypting traffic while still retaining full visibility into application-layer requests.

Another major strength is the deterministic, rule-based detection engine. Unlike machine learning-based systems, Traffic-Sentinel produces predictable and explainable outcomes. Each detection is tied to a clearly defined rule, making auditing and troubleshooting straightforward.

The per-IP reputation scoring mechanism is also a strong feature. Instead of reacting to isolated events, the system accumulates evidence over time, allowing it to distinguish persistent attackers from occasional benign anomalies. This reduces noise and improves alert quality.

Additionally, the project demonstrates strong modularity and integration readiness. Structured event outputs, optional Telegram alerts, and compatibility with external tools such as Fail2Ban or SIEM platforms make Traffic-Sentinel flexible and practical for real deployments.

Finally, the system's safe-by-design philosophy, with no mandatory inline blocking, minimizes the risk of service disruption due to false positives.

8.3 Limitations

Despite its effectiveness, Traffic-Sentinel has several limitations. The most significant limitation is its reliance on predefined detection rules. As a deterministic system, it cannot automatically identify novel or previously unseen attack patterns unless corresponding rules are manually added.

The system also operates in a post-event detection mode, meaning that malicious requests are identified only after they are logged. While this design choice improves safety and simplicity, it does not provide immediate request blocking.

Traffic-Sentinel's detection accuracy is highly dependent on the quality and completeness of web server logs. Misconfigured logging, missing headers, or incorrect client IP resolution (e.g., improper handling of X-Forwarded-For) can reduce effectiveness.

Finally, the current implementation is best suited for small to medium traffic volumes. While performance is efficient, very high-traffic environments may require horizontal scaling or further optimization.

8.4 Recommendations for Future Development

Future versions of Traffic-Sentinel could benefit from several enhancements. Introducing rule versioning and severity weighting would allow more granular control over scoring and alert thresholds.

A lightweight anomaly detection layer could be added alongside the rule engine to identify unusual traffic patterns that do not match known signatures, while still preserving explain ability.

Integration with additional alerting and visualization platforms, such as Slack, email, or a web-based dashboard, would improve usability and situational awareness. Automated firewall enforcement hooks, with configurable confidence thresholds and cooldown mechanisms, could be introduced as an optional feature for environments requiring faster mitigation.

Finally, performance could be improved through parallel log processing or asynchronous event handling to better support higher traffic loads.

8.5 Reflection on Design & Implementation Decisions

The design and implementation choices made in Traffic-Sentinel were well justified given the project's objectives and constraints. The decision to use log-based analysis instead of packet inspection simplified deployment, avoided encryption challenges, and aligned with real-world server architectures.

Choosing a rule-based detection approach ensured transparency, predictability, and low resource usage—key requirements for an academic project focused on explain ability and reliability. While this limits adaptability, it provides strong control and auditability.

The choice to avoid automatic inline blocking reduced operational risk and prevented false positives from disrupting legitimate users. Delegating enforcement to external tools preserved flexibility and system safety.

Technology selections such as Python, Nginx JSON logs, Docker, and Telegram were pragmatic and supported rapid development, maintainability, and portability.

Overall, these decisions reflect a deliberate focus on simplicity, clarity, and practical security value, resulting in a system that fulfills its intended purpose effectively within the project scope.

Chapter 9

Life-long Learning Impact

9.1 Technical & Professional Skills Acquired

The development of **Traffic-Sentinel** resulted in the acquisition of several long-term technical and professional competencies that are directly applicable to real-world software engineering and cybersecurity practice.

Log-Based Security Analysis

A primary technical skill developed during this project is log-centric security analysis. The developer gained hands-on experience in analyzing structured web server access logs (JSONL format) to detect malicious HTTP and HTTPS activity. This included interpreting request metadata such as URI paths, query strings, HTTP status codes, headers, and timestamps to infer attack behavior. The ability to derive actionable security insights purely from application-layer logs reflects an industry-relevant approach used in modern SOC (Security Operations Center) environments.

Rule Engineering and Pattern-Based Threat Detection

The project strengthened expertise in deterministic rule-based detection, particularly through the design and optimization of regular expression (regex) rules. The developer learned how to translate abstract attack concepts (e.g., SQL injection, path traversal, webshell probing) into precise, high-signal detection patterns. Significant emphasis was placed on balancing detection coverage with false-positive reduction, reinforcing disciplined rule engineering skills that are essential in signature-based security systems.

IP Reputation Modeling and Risk Scoring

Another major competency gained is the implementation of an IP reputation and scoring mechanism. The developer learned how to aggregate repeated malicious behaviors over time into a cumulative risk score, enabling differentiation between isolated anomalies and persistent attackers. This mirrors real-world threat intelligence and reputation-based defense strategies, and enhanced the developer's understanding of behavioral correlation and longitudinal security analysis.

Structured Event Logging and Auditability

The project required the generation of structured security event logs in JSON format. Through this, the developer adopted industry best practices in machine-readable logging, ensuring that detection outputs are suitable for downstream consumption by SIEM platforms, alerting systems, or automated enforcement tools. This experience reinforced the importance of auditability, traceability, and standardized data formats in professional software systems.

Modular Software Architecture and Maintainability

From a software engineering perspective, the developer gained strong proficiency in modular system design. Traffic-Sentinel was intentionally architected with clearly separated components for ingestion, normalization, detection, scoring, alerting, and enforcement. This modular approach improved maintainability and extensibility, and helped internalize core architectural principles such as separation of concerns and loose coupling.

Performance Awareness and Optimization

The project also developed practical skills in performance analysis and optimization, particularly in the context of regex-heavy processing. The developer learned to identify performance bottlenecks, refine inefficient patterns, and understand the computational implications of regex backtracking. Exposure to deterministic regex engines (such as RE2) further strengthened the developer's appreciation of algorithmic efficiency and defensive coding practices.

Professional Development Practices

Beyond technical implementation, the project reinforced essential professional skills including version control discipline, technical documentation, and CLI tool development. The developer maintained the project using Git with meaningful commit history, produced comprehensive documentation (README and in-code comments), and built a user-friendly command-line interface. These practices reflect industry-standard workflows and contribute to long-term professional readiness.

Overall, the technical and professional skills acquired through Traffic-Sentinel — spanning security analysis, software architecture, performance engineering, and documentation — represent durable competencies that extend well beyond the scope of this academic project.

9.2 Learning New Technologies & Tools

Adapting to unfamiliar technologies and methodologies was a defining learning experience in the Traffic-Sentinel project. As a solo developer, all exploration, evaluation, and integration of tools were self-directed, reinforcing independent learning and problem-solving skills.

Structured Logging with Nginx and JSONL

One of the most significant learning outcomes was mastering structured JSONL logging in Nginx. Since Nginx does not produce JSON logs by default, the developer learned to configure custom `log_format` directives with proper escaping to generate valid JSON objects per request. This process required careful selection of logged fields and validation of output correctness. Through this, the developer gained practical insight into how structured logging improves parsing efficiency, correlation, and real-time analysis in production systems.

Containerization and Deployment with Docker

The project introduced the developer to Docker-based containerization. Learning how to write Dockerfiles, manage dependencies, and externalize configuration taught key DevOps principles such as environment reproducibility and deployment consistency. Overcoming challenges related to image optimization and configuration management strengthened the developer's confidence in modern deployment workflows.

Third-Party API Integration (Telegram Bot API)

Integrating real-time alerts via Telegram exposed the developer to external API integration for the first time. This involved learning authentication using bot tokens, handling message formatting, and implementing error handling for network failures and rate limits. This experience enhanced the developer's ability to rapidly learn and integrate third-party services — a critical skill in contemporary software development.

Advanced Regex Optimization and Engine Behavior

The project demanded deep learning around regex engine behavior and performance characteristics. Through experimentation and research, the developer learned how certain patterns cause excessive backtracking and how anchoring and constrained quantifiers improve efficiency. Exposure to alternative engines such as RE2 provided a practical lesson in theoretical concepts like NFA vs. DFA matching, grounding abstract theory in real-world application.

Collectively, these learning experiences demonstrate the developer's ability to quickly adapt to new tools, consult official documentation and community resources, and apply newly acquired knowledge effectively under project constraints.

9.3 Future Growth & Directions

Traffic-Sentinel provides a strong foundation for continued technical and professional growth. Several future directions align both with system enhancement and the developer's long-term learning objectives.

Hybrid Detection with Anomaly Analysis

A natural evolution of the project is the integration of anomaly-based detection techniques alongside the existing rule engine. Introducing statistical or machine-learning models to identify deviations from baseline traffic behavior would allow detection of previously unseen threats while complementing deterministic rules. Exploring this area would deepen the developer's expertise in data-driven security and applied machine learning.

Automated Enforcement and Incident Response

Future versions could incorporate controlled, optional real-time enforcement, such as automated firewall rule application based on confidence thresholds. Implementing this safely would strengthen the developer's skills in incident response automation, DevSecOps practices, and risk-aware system design.

Scalability and High-Throughput Processing

As traffic volumes grow, improving scalability through asynchronous I/O, parallel log processing, or distributed deployment would be a valuable technical challenge. Pursuing this direction would expand the developer's understanding of concurrent programming and high-performance system design.

Visualization and Security Dashboards

Developing a web-based dashboard to visualize alerts, IP reputation scores, and attack trends would extend the project into full-stack security tooling. This would enhance the developer's ability to communicate security insights effectively to stakeholders through visual analytics.

In summary, Traffic-Sentinel serves not only as an academic deliverable but as a long-term learning platform. Its future extensions provide a clear pathway for continuous skill development in cybersecurity, software architecture, and advanced system engineering — reinforcing a life-long learning mindset that extends well beyond the scope of this project.

Chapter 10

Conclusion

10.1 Summary of Work

This project presented the design and implementation of **Traffic-Sentinel**, a lightweight, log-driven web traffic detection and security monitoring system. The primary goal was to demonstrate that effective application-layer security analysis can be achieved without inline packet inspection or heavyweight security appliances.

Traffic-Sentinel operates by continuously analyzing structured JSONL access logs generated by web servers and reverse proxies such as Nginx. By leveraging the fact that HTTPS traffic is decrypted at the server or proxy layer, the system retains full visibility into HTTP request details while avoiding the complexity of handling encrypted network packets. Incoming log entries are normalized and evaluated against a curated set of deterministic, rule-based detection patterns covering common web attack vectors including scanning, injection attempts, brute-force authentication, and automated abuse.

The system aggregates detected malicious behavior using a per-IP reputation scoring mechanism, enabling it to distinguish persistent attackers from isolated or low-risk events. All detections are recorded in structured, append-only event logs, ensuring auditability and integration readiness with external security tools. Optional real-time alerting through Telegram further enhances situational awareness without introducing operational risk.

Through a modular architecture, Docker-friendly deployment, and low resource footprint, Traffic-Sentinel achieves a balance between detection capability, explainability, and operational simplicity. The project demonstrates a practical, transparent alternative to traditional web security solutions, particularly suited for small to medium-scale deployments and educational environments.

10.2 Achievement of Objectives

The objectives defined at the outset of the project were successfully achieved, as summarized below:

Malicious Request Detection:

Traffic-Sentinel effectively detects malicious HTTP and HTTPS requests by analyzing structured access logs. The rule-based detection engine identifies a wide range of web attacks, including scanning probes, injection attempts, and automated abuse, fulfilling the core detection objective.

Explainable Rule-Based Analysis:

The system implements a deterministic detection model where every alert is tied to a specific rule match. This ensures transparency, auditability, and ease of interpretation, meeting the objective of explainable security analysis.

IP Reputation and Scoring:

A cumulative per-IP reputation scoring mechanism was successfully implemented. By aggregating

repeated malicious behaviors over time, the system differentiates between occasional anomalies and persistent attackers, improving alert quality and prioritization.

Bruteforce Attack Identification:

Traffic-Sentinel includes logic to detect authentication brute-force attempts by correlating request frequency, targeted endpoints, and HTTP response codes. This enables reliable identification of credential abuse without packet-level inspection.

Structured Security Event Generation:

All detections and actions are recorded in structured JSON-based logs. These logs are suitable for auditing, forensic analysis, and integration with external systems such as SIEM platforms or automated enforcement tools.

Real-Time Alerting Support:

Optional real-time notifications via Telegram were successfully integrated, allowing high-severity events to be delivered instantly to administrators.

Resource-Efficient Operation:

The system was designed and validated to operate with minimal CPU and memory overhead, making it suitable for single-host and small-scale environments.

External Tool Integration:

Traffic-Sentinel produces machine-readable outputs and supports integration with tools such as Fail2Ban, firewalls, and custom automation scripts, achieving its interoperability objective.

Overall, all stated objectives were met within the project scope and constraints, demonstrating the feasibility and effectiveness of the proposed approach.

10.3 Final Remarks

Traffic-Sentinel demonstrates that a log-centric, rule-based security model can provide meaningful and actionable web traffic intelligence without the complexity, opacity, or resource demands of traditional security systems. By prioritizing explainability, modularity, and safe-by-design operation, the project delivers a transparent and practical solution aligned with modern HTTPS-based infrastructures.

From an academic perspective, the project successfully integrates theoretical concepts of web security with real-world system design and implementation practices. It highlights the value of deterministic analysis, structured logging, and reputation-based scoring as effective building blocks for application-layer security monitoring.

Beyond fulfilling course requirements, Traffic-Sentinel serves as a strong foundation for future research and development in cybersecurity. Its extensible architecture allows for enhancements such as anomaly detection, automated enforcement, and visualization dashboards, ensuring continued relevance and learning potential.

In conclusion, Traffic-Sentinel achieves its intended purpose as a lightweight, explainable, and integration-friendly web traffic detection system, while also providing significant educational and practical value. The project reinforces the importance of clarity, reliability, and thoughtful design in building security tools, and prepares the developer for more advanced challenges in software engineering and cybersecurity.

REFERENCES

1. OWASP Foundation, “**OWASP Top 10 Web Application Security Risks,**”
<https://owasp.org/www-project-top-ten/>
2. OWASP Foundation, “**OWASP Core Rule Set (CRS),**”
<https://coreruleset.org/>
3. Vulnerability Payloads, “**swisskyrepo/PayloadsAllTheThings**”
<https://github.com/swisskyrepo/PayloadsAllTheThings/tree/master>
4. Nginx Inc., “**Nginx Official Documentation,**”
<https://nginx.org/en/docs/>
5. Telegram, “**Telegram Bot API Documentation,**”
<https://core.telegram.org/bots/api>

APPENDICES

Appendix A: Project Repository and Source Code

GitHub Repository:

<https://github.com/Arittroskr32/Traffic-Sentinel>

Branch Used: master

Language: Python 3

License: Open-source (educational / academic use)

The repository contains the complete source code, configuration files, Docker setup, deployment scripts, and documentation used in this project.

Appendix B: Nginx JSON Log Format Configuration

Example structured log format used for Traffic-Sentinel:

log_format json_combined escape=json

```
{  
  "ts": "$time_iso8601",  
  "remote_addr": "$remote_addr",  
  "xff": "$http_x_forwarded_for",  
  "host": "$host",  
  "request": "$request",  
  "method": "$request_method",  
  "uri": "$uri",  
  "args": "$args",  
  "status": $status,  
  "bytes": $body_bytes_sent,  
  "ua": "$http_user_agent",  
  "referer": "$http_referer"  
};
```

Appendix C: Sample Detection Rule (rules_custom.yml)

```
- id: SQLI_001  
  category: sql  
  target: args  
  pattern: "(?i)\\b(select|union|where|and|or|sleep|benchmark)\\b"  
  score: 1
```

Appendix D: Sample Security Event Log Output

```
{  
  "timestamp": "2026-01-01T06:53:56Z",  
  "ip": "118.179.120.113",  
  "category": "sqli",  
  "rule_id": "SQLI_001",  
  "score": 1,  
  "uri": "/search_profile",  
  "evidence": "OR '1'='1'--"  
}
```

These logs are append-only and designed for SIEM ingestion and forensic analysis.

Appendix E: Telegram Alert Message Format

TrafficSentinel MARK THRESHOLD REACHED

IP: 118.179.120.113

Score: 10

Categories: sqli, xss

Request: GET /search_profile?username=' OR '1'='1'--

Evidence:

- SQLi pattern matched in args
- XSS payload detected in POST body

Appendix F: Firewall Enforcement Logic (iptables)

Dedicated chain: TS_BLOCK

Rules inserted dynamically

Allowlist respected before banning

Safe unban on expiration

Example:

```
iptables -A TS_BLOCK -s <IP> -j DROP
```

Appendix G: Test Environment Specifications

Cloud Provider: DigitalOcean

VM Specs:

1 vCPU

2 GB RAM

Ubuntu 22.04 LTS

Traffic Source: Real + simulated web requests

Log Rate Tested: ~2,000 requests/sec

Appendix H: Abbreviations

Term	Meaning
IDS	Intrusion Detection System
WAF	Web Application Firewall
SIEM	Security Information and Event Management
XSS	Cross-Site Scripting
SQLi	SQL Injection
LFI	Local File Inclusion
RCE	Remote Code Execution
SSRF	Server-Side Request Forgery
JSONL	JSON Lines
CLI	Command Line Interface